Los Sabrossos

Contents

1 Math **1**
1.1 FFT for Matching . . . 1
1.2 Sieve of Eratosthenes . . . 1
1.3 Euler Phi . . . 1
1.4 FFT for Comparing Rotations . . . 1
1.5 Fft . . . 2
1.6 Binary Exponentiation . . . 3
1.7 Chinese Rest Theorem . . . 3
1.8 Extended Euclidean Algorithm . . . 4
1.9 Least Common Multiple . . . 4
1.10 Logarithmic Fibonacci . . . 5
1.11 Modular Inverse . . . 5
1.12 Pollard Rho and Miller Rabin . . . 5
1.13 Quadroots . . . 6

2 Geometry **6**
2.1 Area of Rectangles . . . 6
2.2 Canonical Line and Perpendicular Line . . . 8
2.3 Circunference . . . 8
2.4 Closest Pair of Points . . . 9
2.5 Closest Pair of Points Lineal . . . 9
2.6 Convex Hull . . . 11
2.7 Line . . . 11
2.8 Line Segment Intersection . . . 12
2.9 Point . . . 12
2.10 Point in Polygon . . . 12
2.11 Polygon . . . 13
2.12 Polygon Convex Area . . . 14
2.13 Polygon Lattice Points . . . 14
2.14 Segments . . . 15
2.15 Stablessum . . . 15

3 Strings **16**
3.1 Kmp . . . 16
3.2 Z Function . . . 16
3.3 Trie . . . 17
3.4 Suffix Array . . . 18
3.5 Aho Corasick . . . 19
3.6 Booths Algorithm . . . 21
3.7 Hashing . . . 22
3.8 Hashing 2d . . . 22
3.9 Manacher . . . 24

4 Bit Manipulation **24**
4.1 Binary Representation . . . 24
4.2 Bit Mask . . . 24
4.3 Turn Off a Bit at Position X . . . 25
4.4 Xor Basis . . . 25

5 Data Structures **26**
5.1 Iterative Segment Tree . . . 26
5.2 Segment Tree . . . 26
5.3 Segment Tree Lazy Propagation . . . 27
5.4 Segment Tree Lazy Struct . . . 28
5.5 1D Binary Indexed Tree . . . 29

5.6 2D Binary Indexed Tree . . . 30
5.7 2D Binary Indexed Tree Infinite Plane . . . 30
5.8 3D Binary Indexed Tree . . . 32
5.9 Disjoint Set Union . . . 32
5.10 Dsu Dynamic Representative . . . 33
5.11 Dynamic Connectivity Offline . . . 33
5.12 Implicit Treap . . . 34
5.13 Mergesort Tree . . . 35
5.14 Ordered Set . . . 36
5.15 Range Bit . . . 37
5.16 Sparse Table . . . 37
5.17 Treap . . . 38

6 Dynamic Programing **39**
6.1 Bounded Knapsack . . . 39
6.2 Digit Dp . . . 39
6.3 Fast Fibonacci . . . 40
6.4 Kadane 2d . . . 41
6.5 Kadane 3d . . . 41
6.6 Longest Increasing Subsequence . . . 42
6.7 Matrix Exponentiation . . . 42
6.8 Sos Dp . . . 42
6.9 Knapsack with Bitset . . . 43

7 Fft **43**
7.1 Ntt . . . 43
7.2 Fft . . . 44
7.3 Fht . . . 45
7.4 Polinomios . . . 47

8 Flow **49**
8.1 Dinic . . . 49

9 Graph Algorithms **50**
9.1 Dijkstra . . . 50
9.2 Kruskal Mst . . . 51
9.3 Topological Sort Kahn Algorithm . . . 51
9.4 Kosaraju . . . 52
9.5 Bridges . . . 52
9.6 Kuhn . . . 53
9.7 2sat . . . 53

10 Tree Algorithms **54**
10.1 Prim . . . 54
10.2 Euler Tour . . . 54
10.3 Binary Lifting . . . 55
10.4 Binary Lifting2 . . . 55
10.5 Centroid Decomposition . . . 55
10.6 Chu Liu Edmonds . . . 57
10.7 Chu Liu Edmonds Construction . . . 58
10.8 Heavy Light Decomposition . . . 60
10.9 Lowest Common Ancestor . . . 61

11 Utilities **62**
11.1 Prime Numbers . . . 62
11.2 2D Prefix Sum . . . 62
11.3 Ascii Table . . . 62
11.4 Cmp for Sort . . . 63
11.5 Int128 . . . 63
11.6 Merge Sort . . . 64
11.7 Pragma . . . 64
11.8 Priority Queue Min Heap . . . 64
11.9 Python Template . . . 65
11.10Random . . . 65

1. Math

1.1. FFT for Matching

```
void solve(){
    string s, p;
    cin >> s >> p;
    int n = s.size();
    int m = p.size();
    vector<int> ans(n - m + 1);
    for(char c = 'A'; c <= 'Z'; c++){
        vector<int> a(n);
        vector<int> b(m);
        for(int i = 0; i < n; i++)
            if(s[i] == c)
                a[i] = 1;
        for(int i = 0; i < m; i++)
            if(p[i] == c)
                b[m - 1 - i] = 1;
        vector<int> cur = multiply(a, b);
        for(int i = 0; i < n - m + 1; i++)
            ans[i] += cur[i + m - 1];
    }
    for(int i = 0; i < n - m + 1; i++){
        if(ans[i] == m)
            cout << i << ' ';
    }
    cout << '\n';
}
```

1.2. Sieve of Eratosthenes

```
const int N = 1e7;
int sieve[N];
void make(){
    for(int i = 0; i < N; i++) sieve[i] = 1;
    sieve[0] = sieve[1] = 0;
    for (int i = 2; i < N; i++)
    {
        if(sieve[i]){
            for (int j = i*i; j < N; j+=i)
            {
                sieve[j] = 0;
            }
        }
    }
}
```

1.3. Euler Phi

```
//cantidad de coprimos con respecto a n
ll phi(ll n) {
    ll ans = n;
    for (ll p = 2; p * p <= n; p++) {
        if (n % p == 0) {
            ans -= ans / p;
            while (n % p == 0){
                n /= p;
            }
        }
    }
    if (n > 1){
        ans -= ans / n;
    }
    return ans;
}
```

1.4. FFT for Comparing Rotations

```
void solve(){
    int n;
    cin >> n;
    vector<ll> a(n), b(n);
    ll suma = 0, sumb = 0;
    for(int i = 0; i < n; i++){
```

```
        cin >> a[i];
        suma += a[i];
    }
    for(int i = 0; i < n; i++){
        cin >> b[i];
        sumb += b[i];
    }
    vector<ll> aa(2 * n);
    vector<ll> ae(2 * n);
    for(int i = 0; i < n; i++){
        aa[i] = aa[i + n] = a[i];
        ae[i] = ae[i + n] = (a[i] != 0);
    }
    vector<ll> be(n);
    vector<ll> rev_b(n);
    for(int i = 0; i < n; i++){
        be[i] = (b[n - 1 - i] != 0);
        rev_b[i] = b[n - 1 - i];
    }
    vector<ll> conv1 = multiply(aa, be);
    vector<ll> conv2 = multiply(ae, rev_b);
    ll ans = 0;
    for(int rot = 0; rot < n; rot++){
        int pos = n - 1 + rot;
        ll perdida = conv1[pos] + conv2[pos];
        ans = max(ans, suma + sumb - perdida);
    }
    cout << ans << '\n';
}
```

1.5. Fft

```
const double PI = acos(-1);

void fft(vector<complex<double>>& a, bool invert) {
    int n = a.size();
    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1)
            j ^= bit;
        j ^= bit;

        if (i < j)
            swap(a[i], a[j]);
    }
    for (int len = 2; len <= n; len <<= 1) {
        double ang = 2 * PI / len * (invert ? -1 : 1);
        complex<double> wlen(cos(ang), sin(ang));
        for (int i = 0; i < n; i += len) {
            complex<double> w(1);
            for (int j = 0; j < len / 2; j++) {
                complex<double> u = a[i + j];
                complex<double> v = a[i + j + len / 2] * w;
                a[i + j] = u + v;
                a[i + j + len / 2] = u - v;
                w *= wlen;
            }
        }
    }
    if (invert) {
        for (complex<double>& x : a)
            x /= n;
    }
}

vector<int> multiply(vector<int> const& a, vector<int> const& b) {
    vector<complex<double>> fa(a.begin(), a.end());
    vector<complex<double>> fb(b.begin(), b.end());
    int n = 1;
    while (n < a.size() + b.size())
        n <<= 1;
    fa.resize(n);
    fb.resize(n);
    fft(fa, 0);
    fft(fb, 0);
    for (int i = 0; i < n; i++)
        fa[i] *= fb[i];
    fft(fa, 1);
    vector<int> res(n);
    for (int i = 0; i < n; i++)
        res[i] = round(fa[i].real());
    return res;
}

```

```cpp
//ejemplo :

void solve(){
    int n;
    cin >> n;

    vector<int> v(n);
    int mx = 0;

    for(int i = 0; i < n; i++){
        cin >> v[i];
        mx = max(mx, v[i]);
    }

    vector<int> pol(mx + 1);

    for(auto x : v)
        pol[x] = 1;

    pol = multiply(pol, pol);

    int m;
    cin >> m;

    int ans = 0;

    for(int i = 0; i < m; i++){
        int x;
        cin >> x;

        if(x < pol.size() and pol[x] > 0)
            ans++;
    }

    cout << ans << '\n';
}
```

1.6. Binary Exponentiation

```cpp
long long pote(long long a, long long b, long long mod){
    long long ans = 1ll;
    while(b){
        if(b&1) ans = (ans * a)%mod;
        a = (a * a)%mod;
        b>>=1;
    }
    return ans;
}
// a^-1 = a^mod-2 = pote(a,mod-2);
```

1.7. Chinese Rest Theorem

```cpp
//si los modulos son muy grandes 1e18, cambiar por int128 las operaciones
ll extgcd(ll a, ll b, ll &x, ll &y) {
    if (!b) {
        x = 1;
        y = 0;
        return a;
    }

    ll x1, y1;
    ll g = extgcd(b, a % b, x1, y1);

    x = y1;
    y = x1 - (a / b) * y1;

    return g;
}

// x      a (mod m)
// x      b (mod n)
//si retorna {0, -1} -> no solution
pair<ll,ll> CRT(ll a, ll m, ll b, ll n) {
    ll x, y;
    ll g = extgcd(m, n, x, y);

    if ((b - a) % g != 0)
        return {0, -1};

    ll n2 = n / g;

```

```
    ll t = ((b - a) / g * x) % n2;
    if (t < 0) t += n2;

    ll mod = m * n2;
    ll r = (a + m * t) % mod;
    if (r < 0) r += mod;

    return {r, mod};
}

// x     a[i] (mod m[i]) para todo i
//vector de valores y vector de respectivos modulos
pair<ll,ll> CRT(vector<ll> a, vector<ll> m) {
    ll r = a[0];
    ll mod = m[0];

    for (int i = 1; i < a.size(); i++) {
        pair<ll,ll> res = CRT(r, mod, a[i], m[i]);

        if (res.second == -1)
            return {0, -1};

        r = res.first;
        mod = res.second;
    }

    return {r, mod};
}

void solve(){
    vector<ll>a(2);
    vector<ll>m(2);
    repl(i,0,2){
        cin >> a[i] >> m[i];
    }
    pair<ll,ll> ans = CRT(a, m);
    if(ans.second == -1){
        cout << "-1\n";
    }
    else{
        cout << ans.first << "␣" << ans.second << "\n";
    }
}
```

1.8. Extended Euclidean Algorithm

```
//Encuentra
//valores para x e y
//A*x + B*y = gcd(A, B)
ll extgcd(ll a, ll b, ll &x, ll &y){
    if(b == 0){
        x = 1;
        y = 0;
        return a;
    }

    ll x1, y1;
    ll g = extgcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - (a / b) * y1;
    return g;
}

void solve(){
    ll A, B, C;
    cin >> A >> B >> C;
    ll x, y;
    ll g = extgcd(A, B, x, y);
    if(C % g != 0){
        cout << "-1\n";
        return;
    }
    ll k = -C / g;
    x *= k;
    y *= k;
    cout << x << "␣" << y << '\n';
}
```

1.9. Least Common Multiple

```cpp
ll lcm(ll a, ll b){
    return (b/__gcd(a,b)) * a;
}
```

1.10. Logarithmic Fibonacci

```cpp
long long int fibo(int n) {
    long long int a, b, x, y, tmp;
    a = x = tmp = 0;
    b = y = 1;
    while (n != 0)
        if (n % 2 == 0) {
            tmp = a * a + b * b;
            b = b * a + a * b + b * b;
            a = tmp;
            n = n / 2;
        } else {
            tmp = a * x + b * y;
            y = b * x + a * y + b * y;
            x = tmp;
            n = n - 1;
        }
    // x = fibo(n-1)
    return y; // y = fibo(n)
}
```

1.11. Modular Inverse

```cpp
const long long N = 1e6 + 10;
const int MOD = 1e9+7;
long long fact[N];
long long invfact[N];
void init(){
    fact[0] = 1;
    for(long long i = 1; i < N; i++)
        fact[i] = fact[i-1] * i % MOD;

    invfact[N-1] = pote(fact[N-1], MOD-2);

        for(long long i = N-2; i >= 0; i--)
        invfact[i] = invfact[i+1] * (i+1) % MOD;
}

long long nCk(long long n, long long k){
    if(k < 0 || k > n) return 0;
    return fact[n] * invfact[k] % MOD * invfact[n-k] % MOD;
}
```

1.12. Pollard Rho and Miller Rabin

```cpp
map<ll, ll> fact;

ll pote(ll a, ll b, ll mod) {
    ll ans = 1;
    while (b) {
        if (b & 1) ans = (__int128)ans * a % mod;
        a = (__int128)a * a % mod;
        b >>= 1;
    }
    return ans;
}

ll mcd(ll a, ll b) {
    a = abs(a);
    b = abs(b);
    while (a != 0) {
        ll r = b % a;
        b = a;
        a = r;
    }
    return b;
}

//Rabin Miller
bool primo(ll n) {
    if (n < 2) return false;
    if (n == 2) return true;
```

```
    ll D = n - 1, S = 0;
    while (D % 2 == 0) {
        D /= 2;
        S++;
    }
    static const ll STEPS = 16ll;
    for(ll pasos = 0; pasos < STEPS; pasos++){
        const ll A = 1 + rand() % (n - 1);
        ll M = pote(A, D, n);
        if (M == 1 || M == (n - 1)) goto next;
        for(ll k = 0; k < S-1; k++){
            M = (__int128)M * M % n;
            if (M == (n - 1)) goto next;
        }
        return false;
    next:;
    }
    return true;
}

//Rho de pollard
ll factor(ll n) {
    static ll A, B;
    A = 1 + rand() % (n - 1);
    B = 1 + rand() % (n - 1);
    #define fun(x) (((__int128)(x) * ((x) + B)) % n + A)

    ll x = 2, y = 2, d = 1;
    while (d == 1 || d == -1) {
        x = fun(x);
        y = fun(fun(y));
        d = mcd(x - y, n);
    }
    return abs(d);
}

void factorize(ll n){
    assert(n > 0);
    while (n > 1 && !primo(n)) {
        ll fx;
        do {
            fx = factor(n);
        } while (fx == n);

        n /= fx;
        factorize(fx);

        for (auto &it : fact) {
            while (n % it.first == 0) {
                n /= it.first;
                it.second++;
            }
        }
    }
    if (n > 1) fact[n]++;
}
```

1.13. Quadroots

```
int quadRoots(double a, double b, double c, pair<double,double> &out){
        assert(a != 0);
        double disc = b*b - 4*a*c;
        if (disc < 0) return 0;
        double sum = (b >= 0) ? -b-sqrt(disc) : -b+sqrt(disc);
        out = {sum/(2*a), sum == 0 ? 0 : (2*c)/sum};
        return 1 + (disc > 0);
}
```

2. Geometry

2.1. Area of Rectangles

```
const ll N = 2e6 + 10;
ll n;
struct info{
    ll mn, cnt;
};
info ST[4 * N];
```

```
ll lazy[4 * N];
info ope(info u, info v){
    if(u.mn < v.mn) return u;
    if(v.mn < u.mn) return v;
    return {u.mn, u.cnt + v.cnt};
}
void build(ll in = 1, ll L = 0, ll R = n - 1){
    lazy[in] = 0;
    if(L == R){
        ST[in] = {0,1};
    }
    else{
        ll mid = (L + R) / 2;
        build(2 * in, L, mid);
        build(2 * in + 1, mid + 1, R);
        ST[in] = ope(ST[2 * in], ST[2 *in + 1]);
    }
}
void push(ll in, ll L, ll R){
    if(lazy[in] == 0){
        return;
    }
    ST[in].mn += lazy[in];

    if(L != R){
        lazy[2 * in] += lazy[in];
        lazy[2 * in + 1] += lazy[in];
    }
    lazy[in] = 0;
}
void update(ll l, ll r, ll val, ll in = 1, ll L = 0, ll R = n - 1){
    push(in, L, R);
    if(r < L or R < l){
        return;
    }
    if(l <= L and R <= r){
        lazy[in] += val;
        push(in, L, R);
        return;
    }
    ll mid = (L + R) / 2;
    update(l, r, val, 2 * in, L, mid);
    update(l, r, val, 2 * in + 1, mid + 1, R);
    ST[in] = ope(ST[2 * in], ST[2 * in + 1]);
}
info get(ll l, ll r, ll in = 1, ll L = 0, ll R = n - 1){
    push(in, L, R);
    if(r < L or R < l){
        return {INT_MAX, 0};
    }
    if(l <= L and R <= r){
        return ST[in];
    }
    ll mid = (L + R) / 2;
    info left = get(l, r, 2 * in, L, mid);
    info right = get(l, r, 2 * in + 1, mid + 1, R);
    return ope(left, right);
}
void solve(){
    ll m; cin >> m;
    vector<array<ll,4>> v;
    repl(i,0,m){
        ll x1,y1,x2,y2;
        cin >> x1 >> y1 >> x2 >> y2;
        v.push_back({x1,y1,y2,1});
        v.push_back({x2,y1,y2,0});
    }
    n = 2e6 + 10;
    build();
    sort(v.begin(),v.end());
    ll last = v[0][0];
    ll ans = 0ll;
    ll off = 1e6;
    repl(i,0,v.size()){
        info q = get(0,n-1);
        ll total = n;
        if(q.mn == 0){
            total = n - q.cnt;
        }
        ll base = v[i][0] - last;
        ans += (total * base);
        last = v[i][0];
        if(v[i][3] == 1){
            update(off + v[i][1], off + v[i][2]-1, 1);
        }
        else{
```

```
            update(off + v[i][1], off + v[i][2]-1, -1);
        }

    }
    cout << ans << "\n";
}
```

2.2. Canonical Line and Perpendicular Line

```
array<ll, 3> get_canonical(ll px, ll py, ll qx, ll qy){
    ll a = 2 * (qx - px);
    ll b = 2 * (qy - py);
    ll c = px*px + py*py - qx*qx - qy*qy;
    ll g = gcd(a, gcd(b, c));
    a /= g; b /= g; c /= g;
    if(a < 0) a = -a, b = -b, c = -c;
    if(a == 0 and b < 0) b = -b, c = -c;
    return {a,b,c};
}
void solve(){
    ll px,py,qx,qy,rx,ry,sx,sy;
    cin >> px >> py >> qx >> qy >> rx >> ry >> sx >> sy;
    array<ll, 3> uwu = get_canonical(px,py,qx,qy);
    array<ll, 3> owo = get_canonical(rx,ry,sx,sy);
    if(uwu[0]*owo[1] == owo[0]*uwu[1]){ // is perpendicular
        if(uwu == owo){ // same line
            cout << "Yes\n";
        }
        else{
            cout << "No\n";
        }
    }
    else{
        cout << "Yes\n";
    }
}
```

2.3. Circunference

```
pt circumCenter(pt a, pt b, pt c) {
        b = b-a, c = c-a; // consider coordinates relative to A
        assert(cross(b,c) != 0); // no circumcircle if A,B,C aligned
        return a + perp(b*sq(c) - c*sq(b))/cross(b,c)/2;
}

int circleLine(pt o, double r, line l, pair<pt,pt> &out) {
        double h2 = r*r - l.sqDist(o);
        if (h2 >= 0) { // the line touches the circle
                pt p = l.proj(o); // point P
                pt h = l.v*sqrt(h2)/abs(l.v); // vector parallel to l, of
                length h
                out = {p-h, p+h};
        }
        return 1 + sgn(h2);
}

int circleCircle(pt o1, double r1, pt o2, double r2, pair<pt,pt> &out) {
        pt d=o2-o1; double d2=sq(d);
        if (d2 == 0) {assert(r1 != r2); return 0;} // concentric circles
        double pd = (d2 + r1*r1 - r2*r2)/2; // = |O_1P| * d
        double h2 = r1*r1 - pd*pd/d2; // = h 2
        if (h2 >= 0) {
                pt p = o1 + d*pd/d2, h = perp(d)*sqrt(h2/d2);
                out = {p-h, p+h};
        }
        return 1 + sgn(h2);
}

int tangents(pt o1, double r1, pt o2, double r2, bool inner, vector<pair<pt,pt>> &out) {
        if (inner) r2 = -r2;
        pt d = o2-o1;
        double dr = r1-r2, d2 = sq(d), h2 = d2-dr*dr;
        if (d2 == 0 || h2 < 0) {assert(h2 != 0); return 0;}
        for (double sign : {-1,1}) {
                pt v = (d*dr + perp(d)*sqrt(h2)*sign)/d2;
                out.push_back({o1 + v*r1, o2 + v*r2});
        }
        return 1 + (h2 > 0);
}
```

2.4. Closest Pair of Points

```
struct point{
    long long x,y;
};
long long sq(long long a){
    return a * a;
}
long long distance(point a, point b){
    long long A = sq((a.x - b.x));
    long long B = sq((a.y - b.y));
    return A + B;
}
long long closest(vector<point>& X, vector<point>& Y){
    int n = X.size();
    if(n <= 1){
        return LLONG_MAX;
    }
    vector<point> left_x = vector<point>(X.begin(), X.begin() + (n / 2));
    vector<point> right_x = vector<point>(X.begin() + (n / 2), X.end());
    vector<point> left_y, right_y;
    for(point p : Y){
        if(make_pair(p.x, p.y) <= make_pair(left_x.back().x, left_x.back().y)){
            left_y.push_back(p);
        }
        else{
            right_y.push_back(p);
        }
    }
    long long d1 = closest(left_x, left_y);
    long long d2 = closest(right_x, right_y);
    long long d = min(d1,d2);
    long long mid_x = left_x.back().x;
    vector<point> stripe;
    for(point& p : Y){
        if(sq(p.x - mid_x) < d){
            stripe.push_back(p);
        }
    }
    for(int i = 0; i < (int) stripe.size(); i++){
        for(int j = i + 1; j < (int) stripe.size() and sq(stripe[j].y - stripe[i].y) < d; j++){
            d = min(d, distance(stripe[i], stripe[j]));
        }
    }
    return d;
}
void solve(){
    int n; cin >> n;
    vector<point> v(n);
    for(point& x : v){
        cin >> x.x >> x.y;
    }
    sort(v.begin(),v.end(), [&](const point& a, const point& b){
        return make_pair(a.x, a.y) < make_pair(b.x, b.y);
    });
    vector<point> w = v;
    sort(w.begin(),w.end(),[&](const point& a, const point& b){
        return a.y < b.y;
    });
    long long ans = closest(v,w);
    cout << ans << "\n";
}
```

2.5. Closest Pair of Points Lineal

```
struct pt {
    ll x, y;
    bool operator == (const pt & o) const {
        return x == o.x and y == o.y;
    }
};

struct CustomHashPoint {
    size_t operator()(const pt & p) const {
        static
        const uint64_t C =
            chrono::steady_clock::now().time_since_epoch().count();
        uint64_t x = p.x;
        uint64_t y = p.y;
        x ^= x >> 30;
        x *= 0xbf58476d1ce4e5b9;
        x ^= x >> 27;
        x *= 0x94d049bb133111eb;
```

```
        x ^= x >> 31;
        y ^= y >> 30;
        y *= 0xbf58476d1ce4e5b9;
        y ^= y >> 27;
        y *= 0x94d049bb133111eb;
        y ^= y >> 31;
        return C ^ (x + (y << 1));
    }
};

ll dist2(pt a, pt b) {
    ll x = a.x - b.x;
    ll y = a.y - b.y;
    return x * x + y * y;
}

pair < int, int > closest_pair(vector < pt > p) {
    int n = p.size();
    unordered_map < pt, int, CustomHashPoint > mp;
    // puntos repetidos
    for (int i = 0; i < n; i++) {
        if (mp.count(p[i]))
            return {
                mp[p[i]],
                i
            };
        mp[p[i]] = i;
    }
    mt19937 rng(
        chrono::steady_clock::now().time_since_epoch().count()
    );
    auto rnd = [ & ]() {
        return uniform_int_distribution < int > (0, n - 1)(rng);
    };
    ll d2 = dist2(p[0], p[1]);
    pair < int, int > ans = {
        0,
        1
    };
    auto upd = [ & ](int i, int j) {
        ll nd = dist2(p[i], p[j]);
        if (nd < d2) {
            d2 = nd;
            ans = {
                i,
                j
            };
        }
    };
    // conseguir una distancia inicial razonable
    for (int it = 0; it < 20; it++) {
        int i = rnd();
        int j = rnd();
        while (i == j) j = rnd();
        upd(i, j);
    }
    ll d = sqrtl((long double) d2) + 1;
    unordered_map < pt, vector < int > , CustomHashPoint > grid;
    grid.reserve(2 * n);
    for (int i = 0; i < n; i++) {
        grid[{
            p[i].x / d,
            p[i].y / d
        }].push_back(i);
    }
    for (auto & [c, v]: grid) {
        // misma celda
        for (int i = 0; i < (int) v.size(); i++) {
            for (int j = i + 1; j < (int) v.size(); j++) {
                upd(v[i], v[j]);
            }
        }
        // celdas vecinas
        for (int dx = -1; dx <= 1; dx++) {
            for (int dy = -1; dy <= 1; dy++) {
                if (dx == 0 and dy == 0) continue;
                pt to = {
                    c.x + dx,
                    c.y + dy
                };
                auto it = grid.find(to);
                if (it == grid.end()) continue;
                for (int i: v) {
                    for (int j: it -> second) {
                        upd(i, j);
                    }
```

```
                }
            }
        }
    }
    return ans;
}
```

2.6. Convex Hull

```
// Devuelve sentido horario
vector<point> hull(vector<point> p)
{
    int n = p.size();
        vector<point> h;
        sort(p.begin(),p.end());
        for(int i = 0; i < n; i++){
                while(h.size() >= 2 and p[i].left(h[h.size () - 2], h.back()) < 0){
                        h.pop_back();
                }
                h.push_back(p[i]);
        }
        h.pop_back();
        int k = h.size();
        for(int i = n-1; i >= 0; i--)
        {
                while(h.size() >= k + 2 and p[i].left(h[h.size() - 2], h.back()) < 0){
                        h.pop_back();
                }
                h.pb(p[i]);
        }
        h.pop_back();
        return h;
}
```

2.7. Line

```
typedef double T;

struct line {
        pt v; T c;
        // From direction vector v and offset c
        line(pt v, T c) : v(v), c(c) {}
        // From equation ax+by=c
        line(T a, T b, T c) : v({b,-a}), c(c) {}
        // From points P and Q
        line(pt p, pt q) : v(q-p), c(cross(v,p)) {}
        // Will be defined later:
        // - these work with T = int
        T side(pt p);
        double dist(pt p);
        line perpThrough(pt p);
        bool cmpProj(pt p, pt q);
        line translate(pt t);
        // - these require T = double
        void shiftLeft(double dist);
        pt proj(pt p);
        pt refl(pt p);
}

T side(pt p) {return cross(v,p)-c;}

double dist(pt p) {return abs(side(p)) / abs(v);}

double sqDist(pt p) {return side(p)*side(p) / (double)sq(v);}

line translate(pt t) {return {v, c + cross(v,t)};}

line shiftLeft(double dist) {return {v, c + dist*abs(v)};}

bool inter(line l1, line l2, pt &out) {
        T d = cross(l1.v, l2.v);
        if (d == 0) return false;
        out = (l2.v*l1.c - l1.v*l2.c) / d; // requires floating-point
        coordinates
        return true;
}

pt proj(pt p) {return p - perp(v)*side(p)/sq(v);}

pt refl(pt p) {return p - perp(v)*2*side(p)/sq(v);}

```

```
line bisector(line l1, line l2, bool interior) {
        assert(cross(l1.v, l2.v) != 0); // l1 and l2 cannot be parallel!
        double sign = interior ? 1 : -1;
        return {l2.v/abs(l2.v) + l1.v/abs(l1.v) * sign,
        l2.c/abs(l2.v) + l1.c/abs(l1.v) * sign};
}
```

2.8. Line Segment Intersection

```
bool cruzan(point a, point b, point c, point d) {
    ll x = (b-a) ^ (c-a);
    ll y = (b-a) ^ (d-a);
    ll z = (d-c) ^ (a-c);
    ll w = (d-c) ^ (b-c);

    if (((x > 0) != (y > 0)) &&
        ((z > 0) != (w > 0)))
        return true;

    if (x == 0 && on_segment(a,b,c)) return true;
    if (y == 0 && on_segment(a,b,d)) return true;
    if (z == 0 && on_segment(c,d,a)) return true;
    if (w == 0 && on_segment(c,d,b)) return true;

    return false;
}
```

2.9. Point

```
struct point
{
        int x, y;
        point() {}
        point(int x, int y): x(x), y(y) {}
        point operator -(point p) {return point(x - p.x, y - p.y);}
        point operator +(point p) {return point(x + p.x, y + p.y);}
        int sq() const{return x * x + y * y;}
        double abs() const {return sqrt(sq());}
        int operator ^(point p) {return x * p.y - y * p.x;}
        int operator *(point p) {return x * p.x + y * p.y;}
        point operator *(int a) {return point(x * a, y * a);}
        bool operator <(const point& p) const {return x == p.x ? y < p.y : x < p.x;}
        int left(point a, point b) {
        return ((b - a) ^ (*this - a)); // 0, -, +
    }
};
ostream& operator<<(ostream& os, const point& p) {
    return os << "(" << p.x << "," << p.y << ")\n";
}


void polarSort(vector<point>& v) {
  sort(v.begin(), v.end(), [] (point a, point b) {
    const point origin{0, 0};
    bool ba = a < origin, bb = b < origin;
    if (ba != bb) { return ba < bb; }
    return (a^b) > 0;
  });
}

double tama(point x){
        return sqrt(x.x*x.x + x.y+x.y);
}

double angle(point v, point w){
        double cosTheta = v*w / tama(v) / tama(w);
        return acos(max(-1,0, min(1,0, cosTheta)));
}
```

2.10. Point in Polygon

```
bool cruzan(point a, point b, point c, point d) {
    double x = (b-a) ^ (c-a);
    double y = (b-a) ^ (d-a);
    double z = (d-c) ^ (a-c);
    double w = (d-c) ^ (b-c);

```

```
    if ((x > 0 && y > 0) || (x < 0 && y < 0)) return false;

    if (x == 0 && y == 0) {
        if (a < c && a < d && b < c && b < d) return false;
        if (c < a && c < b && d < a && d < b) return false;
        return true;
    }

    return !((z > 0 && w > 0) || (z < 0 && w < 0));
}

bool estamid(point p, point a, point b) {
    if (b < a) swap(a, b);
    return p.left(a, b) == 2 && a < p && p < b;
}

void solve() {
    ll n, m;
    cin >> n >> m;

    vector<point> v(n);
    repl(i, 0, n) cin >> v[i].x >> v[i].y;

    repl(i, 0, m) {
        point p;
        cin >> p.x >> p.y;

        double a = (rand() % 9 + 1) / 10.0;
        double b = (rand() % 9 + 1) / 10.0;
        point q(p.x + b, 1e18 + a);

        ll cnt = 0;
        bool boundary = false;

        repl(j, 0, n) {
            ll k = (j + 1) % n;

            cnt += cruzan(p, q, v[j], v[k]);

            if (estamid(p, v[j], v[k]) ||
                p.x == v[j].x && p.y == v[j].y ||
                p.x == v[k].x && p.y == v[k].y)
                boundary = true;
        }

        if (boundary) cout << "BOUNDARY\n";
        else if (cnt & 1) cout << "INSIDE\n";
        else cout << "OUTSIDE\n";
    }
}
```

2.11. Polygon

```
double areaTriangle(pt a, pt b, pt c) {
        return abs(cross(b-a, c-a)) / 2.0;
}

double areaPolygon(vector<pt> p) {
        double area = 0.0;
        for (int i = 0, n = p.size(); i < n; i++) {
                area += cross(p[i], p[(i+1)%n]);
                // wrap back to 0 if i == n - 1
        }
        return abs(area) / 2.0;
}

bool above(pt a, pt p) {
        return p.y >= a.y;
}
// check if [PQ] crosses ray from A
bool crossesRay(pt a, pt p, pt q) {
        return (above(a,q) - above(a,p)) * orient(a,p,q) > 0;
}

bool inPolygon(vector<pt> p, pt a, bool strict = true) {
        int numCrossings = 0;
        for (int i = 0, n = p.size(); i < n; i++) {
                if (onSegment(p[i], p[(i+1)%n], a))
                return !strict;
                numCrossings += crossesRay(a, p[i], p[(i+1)%n]);
        }
        return numCrossings & 1; // inside if odd number of crossings
}

```

```
double angleTravelled(pt a, pt p, pt q) {
        double ampli = angle(p-a, q-a);
        if (orient(a,p,q) > 0) return ampli;
        else return -ampli;
}

double angleTravelled(pt a, pt p, pt q) {
        // remainder ensures the value is in [-pi,pi]
        return remainder(arg(q-a) - arg(p-a), 2*M_PI);
}

int windingNumber(vector<pt> p, pt a) {
        double ampli = 0;
        for (int i = 0, n = p.size(); i < n; i++)
                ampli += angleTravelled(a, p[i], p[(i+1)%n]);
        return round(ampli / (2*M_PI));
}

struct angle {
        pt d; int t = 0; // direction and number of full turns  angle t180();

        angle t360() {return {d, t+1};}
        angle t180() {return {d*(-1), t + half(d)};}

        bool operator<(angle a, angle b) {
                return make_tuple(a.t, half(a.d), 0) <
                make_tuple(b.t, half(b.d), cross(a.d,b.d));
        }
};

bool half(pt p) {
        return p.y > 0 || (p.y == 0 && p.x < 0);
}


angle moveTo(angle a, pt newD) {
        // check that segment [ D D  ] d o e s n t  go through the origin
        assert(!onSegment(a.d, newD, {0,0}));
        angle b{newD, a.t};
        if (a.t180() < b) // if b more than half a turn bigger
                b.t--; // decrease b by a full turn
        if (b.t180() < a) // if b more than half a turn smaller
                b.t++; // increase b by a full turn
        return b;
}

int windingNumber(vector<pt> p, pt a) {
        angle a{p.back()}; // start at last vertex
        for (pt d : p)
                a = moveTo(a, d); // move to first vertex, second, etc.
        return a.t;
}
```

2.12. Polygon Convex Area

```
ll area2(vector<point>& p) {
    ll s = 0;
    int n = p.size();

    for (int i = 0; i < n; i++)
        s += p[i] ^ p[(i + 1) % n];

    return abs(s);
}
```

2.13. Polygon Lattice Points

```
pair<ll,ll> pick(vector<point>& v) {
    ll B = 0, A2 = 0;
    ll n = v.size();

    for (ll i = 0; i < n; i++) {
        ll j = (i + 1) % n;
        B += __gcd(abs(v[i].x - v[j].x),
                   abs(v[i].y - v[j].y));
    }

    for (ll i = 2; i < n; i++)
        A2 += (v[i] - v[0]) ^ (v[i-1] - v[0]);

    A2 = abs(A2);
```

```
15
16      ll I = (A2 - B + 2) / 2;
17
18      return {I, B};
19  }
20  // el poligono no debe intersectarse, puede NO ser convexo y los puntos en el vector deben ser
        adyacentes.
```

2.14. Segments

```
1   bool inDisk(pt a, pt b, pt p) {
2           return dot(a-p, b-p) <= 0;
3   }
4
5   bool onSegment(pt a, pt b, pt p) {
6           return orient(a,b,p) == 0 && inDisk(a,b,p);
7   }
8
9   bool properInter(pt a, pt b, pt c, pt d, pt &out) {
10          double oa = orient(c,d,a),
11                  ob = orient(c,d,b),
12                  oc = orient(a,b,c),
13                  od = orient(a,b,d);
14          // Proper intersection exists iff opposite signs
15          if (oa*ob < 0 && oc*od < 0) {
16                  out = (a*ob - b*oa) / (ob-oa);
17                  return true;
18          }
19          return false;
20  }
21
22  struct cmpX {
23          bool operator()(pt a, pt b) {
24                  return make_pair(a.x, a.y) < make_pair(b.x, b.y);
25          }
26  };
27  set<pt,cmpX> inters(pt a, pt b, pt c, pt d) {
28          pt out;
29          if (properInter(a,b,c,d,out)) return {out};
30          set<pt,cmpX> s;
31          if (onSegment(c,d,a)) s.insert(a);
32          if (onSegment(c,d,b)) s.insert(b);
33          if (onSegment(a,b,c)) s.insert(c);
34          if (onSegment(a,b,d)) s.insert(d);
35          return s;
36  }
37
38  double segPoint(pt a, pt b, pt p) {
39          if (a != b) {
40          line l(a,b);
41          if (l.cmpProj(a,p) && l.cmpProj(p,b)) // if closest to
42                  projection
43                  return l.dist(p); // output distance to
44                  line
45          }
46          return min(abs(p-a), abs(p-b)); // otherwise distance to A or B
47  }
48
49  double segSeg(pt a, pt b, pt c, pt d) {
50          pt dummy;
51          if (properInter(a,b,c,d,dummy))
52          return 0;
53          return min({segPoint(a,b,c), segPoint(a,b,d),
54          segPoint(c,d,a), segPoint(c,d,b)});
55  }
```

2.15. Stablessum

```
1   struct stableSum {
2           int cnt = 0;
3           vector<double> v, pref{0};
4           void operator+=(double a) {
5                   assert(a >= 0);
6                   int s = ++cnt;
7                   while (s % 2 == 0) {
8                           a += v.back();
9                           v.pop_back(), pref.pop_back();
10                          s /= 2;
11                  }
12                  v.push_back(a);
13                  pref.push_back(pref.back() + a);
```

```
        }

        double val() {return pref.back();}
};
```

3. Strings

3.1. Kmp

```
vector<int> kmp(string s){
    int n = s.size();
    vector<int> pi(n);
    for(int i = 1; i < n; i++){
        int j = pi[i-1];
        while(j > 0 and s[i] != s[j]){
            j = pi[j-1];
        }
        if(s[i] == s[j]){
           j++;
        }
        pi[i] = j;
    }
    return pi;
}
//Pattern Matching
vector<int> ocurrencia(string texto, string patron) {
    vector<int> P = kmp(patron);
    int k = 0;
    vector<int> M;
    repl(i,0,texto.size()) {
        while (k > 0 and patron[k] != texto[i]) k = P[k - 1];
        if (patron[k] == texto[i]) k++;
        if (k == patron.size()) {
            M.pb(i - k + 1);
            k = P[k - 1];
        }
    }
    return M;
}

//Automata
void genAut(string &s) {
    int n = s.size();

    vector<vector<int>> aut(n+1, vector<int>(26));
    vector<int> pi = kmp(s);

    for(int i = 0 ; i < n; i++){
        aut[i][s[i]-'a'] = i+1;
    }

    for(int i = 0; i < n+1; i++){
        for(int c = 0; c < 26; c++){
            if (aut[i][c]) continue;
            if (i < n and s[i] == 'a' + c) aut[i][c] = i+1;
            else if (i > 0) aut[i][c] = aut[pi[i-1]][c];
            else aut[i][c] = 0;
        }
    }
}
```

3.2. Z Function

```
vector<int> z_function(string s){
    int n = s.size();
    vector<int> z(n,0);
    int l = 0;
    int r = 0;
    repl(i,1,n){
        if(i < r){
            z[i] = min(r - i, z[i - l]);
        }
        while(i + z[i] < n and s[z[i]] == s[i + z[i]]){
            z[i]++;
        }
        if(i + z[i] > r){
            l = i;
            r = i + z[i];
```

```
        }
    }
    return z;
}
```

3.3. Trie

```
const int E = 26;

struct AC {
    int nodes;
    vector<array<int, E>>go;
    vector<bool>terminal;

    AC(){
        add_node();
        nodes = 1;
    }

    void add_node(){
        terminal.emplace_back();
        go.emplace_back();
    }

    void insert(string &s){
        int pos =  0;
        for(char ch : s){
            int c = ch - '0';
            if(go[pos][c] == 0){
                go[pos][c] = nodes++;
                add_node();
            }
            pos = go[pos][c];
        }
        terminal[pos] =  true;
    }

};
//TRIE XOR
struct Trie{
    int nodes;
    vector<array<int,E>> go;
    vector<int> can;
    Trie(){
        add_node();
        nodes = 1;
    }
    void add_node(){
        go.push_back({-1,-1});
        can.emplace_back();
    }
    void insert(int x){
        int pos = 0;
        can[pos]++;
        for(int i = 29; i >= 0; i--){
            int bit = ((x >> i) & 1);
            if(go[pos][bit] == -1){
                add_node();
                go[pos][bit] = nodes++;
            }
            pos = go[pos][bit];
            can[pos]++;
        }
    }
    void erase(int x){
        int pos = 0;
        can[pos]--;
        for(int i = 29; i >= 0; i--){
            int bit = ((x >> i) & 1);
            int next = go[pos][bit];
            can[next]--;
            if(can[next] == 0) go[pos][bit] = -1;
            pos = next;
        }
    }
    int get(int x, int k){
        int val = 0;
        int pos = 0;
        for(int i = 29; i >= 0; i--){
            int b = (x >> i) & 1;
            int nxt = go[pos][b];
            if(nxt != -1 and can[nxt] >= k){
                pos = nxt;
                val += (1 << i) * b;
```

```
            }
            else{
                if(nxt != -1) k -= can[nxt];
                pos = go[pos][b ^ 1];
                val += (1 << i) * (b ^ 1);
            }
        }
        return val ^ x;
    }
};
```

3.4. Suffix Array

```
vector<int> suffix_array(string s) {
    s += char(0);
    const int n = s.size();
    vector<int>a(n);
    iota(a.begin(), a.end(), 0);
    sort(a.begin(), a.end(), [&] (int i, int j){
        return s[i] < s[j];
    });
    vector<int> mapping(n);
    mapping[a[0]] = 0;
    for(int i = 1; i < n; i++){
        mapping[a[i]] = mapping[a[i-1]]+(s[a[i-1]] != s[a[i]]);
    }
    int len = 1;
    vector<int>sbs(n);
    vector<int>head(n);
    vector<int>new_mapping(n);
    while(len < n){
        for(int i = 0; i<n; i++) sbs[i] = (a[i] + n - len) % n;
        for(int i = n-1; i>=0; i--) head[mapping[a[i]]] = i;
        for(int i = 0; i < n; i++){
            int x = sbs[i];
            a[head[mapping[x]]++] = x;
        }
        new_mapping[a[0]] = 0;
        for(int i = 1; i < n; i++){
            if(mapping[a[i-1]] != mapping[a[i]]){
                new_mapping[a[i]] = new_mapping[a[i-1]]+1;
            }
            else{
                int pre = mapping[(a[i-1] + len)%n];
                int cur = mapping[(a[i]+len) % n];
                new_mapping[a[i]] = new_mapping[a[i-1]] + (pre!=cur);
            }
        }
        len <<= 1;
        swap(mapping, new_mapping);
    }
    return vector<int>(a.begin()+1, a.end());//ignorar a[0] que es el centinela
}

vector<int> lcp_array(vector<int>&a, string s){
    const int n = s.size();
    vector<int> rank(n);
    for(int i = 0; i < n; i++) rank[a[i]] = i;
    int k = 0;
    vector<int>lcp(n);
    for(int i = 0; i < n; i++){
        if(rank[i] == n-1){
            k = 0;
            continue;
        }
        int j = a[rank[i]+1];
        while(i+k < n and j+k < n and s[i+k] == s[j+k]) k++;
        lcp[rank[i]] = k;
        if(k) k--;
    }
    lcp.pop_back();
    return lcp;
}

void solve(){
    //Numero de distintos substr en un string n*(n+1)/2 - sumatoria del LCP
    //verificar si un string es substring de otro con BS en SA
    //encontrar el longest common substring en 2 strings(ESTA IMPLEMENTACION)
    string s, t;
    cin >> s >> t;
    if(s.size() > t.size()){
        swap(s, t);
    }
    int n = s.size();
```

```
    int m = t.size();
    string st = s+'#'+t;
    vector<int>sa = suffix_array(st);
    vector<int>lcp =lcp_array(sa, st);
    // print(sa);
    // print(lcp);
    vector<int>tr(n+m);
    repl(i,0,n+m){
        if(sa[i] >=n and sa[i+1]>=n){
            tr[i] = 0;
            continue;
        }
        if(sa[i] < n and sa[i+1] < n){
            tr[i] = 0;
            continue;
        }
        int mn = min(sa[i], sa[i+1]);
        int tam1 = mn + lcp[i];
        // dbg(tam1);
        tr[i] = lcp[i];
        if(tam1 > n){
            dbg(tam1);
            tr[i] = lcp[i] - (tam1-n);
        }
    }
    // print(tr);
    int mx = *max_element(tr.begin(), tr.end());
    repl(i,0,n+m){
        // dbg(tr[i]);
        if(tr[i] == mx){
            string ans = st.substr(sa[i], mx);
            cout << ans << '\n';
            return;
        }
    }

}
//Sabrossus
```

3.5. Aho Corasick

```
//AhoCorasick para saber si un string s existe en un texto T
const int E = 26;
const int N = 1e6+10;
int cur;
int nodoTerminal[N];//se guarda el nodo terminal del string i
set<int>st;//se guarda que nodos son terminales que aparecen en el texto T

struct AC {
    int nodes;
    vector<int>suf, super;
    vector<array<int, E>>go;
    vector<bool>terminal;
    AC(){
        add_node();
        nodes = 1;
    }
    void add_node(){
        terminal.emplace_back();
        go.emplace_back();
        super.emplace_back();
        suf.emplace_back();
    }

    void insert(string &s){
        int pos =  0;
        for(char ch : s){
            int c = ch - 'a';
            if(go[pos][c] == 0){
                go[pos][c] = nodes++;
                add_node();
            }
            pos = go[pos][c];
        }
        terminal[pos] =  true;
        nodoTerminal[cur] = pos;
    }

    void build(){
        queue<int>Q;
        Q.emplace(0);
        while(!Q.empty()){
            int u = Q.front();
            Q.pop();
```

```
            super[u] = (suf[u] == 0 or terminal[suf[u]] ? suf[u] : super[suf[u]]);
            for(int c = 0; c < E; c++){
                if(go[u][c]){
                    int v = go[u][c];
                    Q.emplace(v);
                    suf[v] = u == 0 ? 0 : go[suf[u]][c];
                }
                else{
                    go[u][c] = u == 0 ? 0 :go[suf[u]][c];
                }
            }
        }
    }
};

void mark(int u, AC &ac){
    if(u == 0) return;
    mark(ac.super[u], ac);
    if(ac.terminal[u]){
        st.insert(u);
        ac.terminal[u] = false;
    }
    ac.super[u] = 0;
}

void solve(){
    int n;
    cin >> n;
    AC ac;
    repl(i,0,n){
        string s;
        cin >> s;
        cur = i;
        ac.insert(s);
    }
    string t; //texto grande
    cin >> t;
    ac.build();
    int p = 0;
    for(auto x : t){
        int c = x - 'a';
        p = ac.go[p][c];
        mark(p, ac);
    }
    repl(i,0,n){
        if(st.count(nodoTerminal[i])){
            cout << "YES\n";
        }
        else{
            cout << "NO\n";
        }
    }
}

//------------------------------------------------------------------------------

//AhoCorasick para Frecuencias
const int E = 26;
const int N = 1e6+10;
int nodoTerminal[N];
int cur;
struct AC {
    int nodes;
    vector<int>suf, freq, by_level;
    vector<bool>terminal;
    vector<array<int,E>>go;

    AC(){
        add_node();
        nodes = 1;
    }
    void add_node(){
        terminal.emplace_back();
        suf.emplace_back();
        go.emplace_back();
        freq.emplace_back();
    }

    void insert(string &s){
        int pos = 0;
        for(char ch:s){
            int c = ch - 'a';
            if(go[pos][c] == 0){
                go[pos][c] = nodes++;
                add_node();
            }
```

```
            pos = go[pos][c];
        }
        terminal[pos] = true;
        nodoTerminal[cur] = pos;
    }

    void build(){
        queue<int>Q;
        Q.emplace(0);
        while(!Q.empty()){
            int u = Q.front();
            Q.pop();
            by_level.emplace_back(u);
            for(int c = 0; c < E; c++){
                if(go[u][c]){
                    int v = go[u][c];
                    Q.emplace(v);
                    suf[v] = u == 0 ? 0 : go[suf[u]][c];
                }
                else{
                    go[u][c] = u == 0 ? 0 : go[suf[u]][c];
                }
            }
        }
    }
};

void solve(){
    int n;
    cin >> n;
    AC ac;
    repl(i,0,n){
        string s;
        cin >> s;
        cur = i;
        ac.insert(s);
    }
    string t;
    cin >> t;
    ac.build();
    int p = 0;
    for(char ch:t){
        int c = ch-'a';
        p = ac.go[p][c];
        ac.freq[p]++;
    }
    for(int i = (int)ac.by_level.size()-1; i > 0; i--){
        int x = ac.by_level[i];
        ac.freq[ac.suf[x]] += ac.freq[x];
    }
    repl(i,0,n){
        int termi = nodoTerminal[i];
        cout << ac.freq[termi] << '\n';
    }
}
```

3.6. Booths Algorithm

```
//Algoritmo para menor rotacion lexicograf ca de un string en O(N)

int menorRotacion(string &s){
    int n = s.size();
    vector<int>f(2*n, -1);
    int k = 0; //indice mejor rotacion
    for(int j = 1; j < 2*n;j++){
        int i = f[j-k-1];
        //el operador %n simula la concatenacion de la cadena s+s
        while(i!=-1 and s[j%n]!=s[(k+i+1)%n]){
            if(s[j%n] < s[(k+i+1)%n]){
                k = j - i - 1;
            }
            i = f[i];
        }
        if(i == -1 and s[j%n] != s[(k+i+1)%n]){
            if(s[j % n] < s[(k+i+1%n)]){
                k = j;
            }
            f[j - k] = -1;
        }
        else{
            f[j - k] = i + 1;
        }
    }
    return k;
```

```
}
void solve(){
    string s;
    cin >> s;
    ll n = s.size();
    ll ans = menorRotacion(s);
    repl(i, ans, ans+n){
        cout << s[i%n];
    }
}
```

3.7. Hashing

```
struct Hash {
        long long P=1777771,MOD[2],PI[2];
        vector<long long> h[2],pi[2];

        // SOLO PARA ROLLING / JOIN:
        vector<long long> pw[2];

        Hash(string& s){
                MOD[0]=999727999;
                MOD[1]=1070777777;
                PI[0]=325255434;
                PI[1]=10018302;

                for(int k = 0; k < 2; k++) {
                        h[k].resize(s.size()+1);
                        pi[k].resize(s.size()+1);

                        // SOLO PARA ROLLING / JOIN:
                        pw[k].resize(s.size()+1);
                }

                for(int k = 0; k < 2; k++){
                        h[k][0] = 0;
                        pi[k][0] = 1;

                        // SOLO PARA ROLLING / JOIN:
                        pw[k][0] = 1;

                        long long p = 1;

            for(int i = 1; i < s.size()+1; i++){
                                h[k][i] = (h[k][i-1]+p*s[i-1])%MOD[k];
                                pi[k][i] = (1LL*pi[k][i-1]*PI[k])%MOD[k];

                                // SOLO PARA ROLLING / JOIN:
                                pw[k][i] = (1LL*pw[k][i-1]*P)%MOD[k];

                                p = (p*P)%MOD[k];
                        }
                }
        }

        long long get(int s, int e){
                long long h0 = (h[0][e]-h[0][s]+MOD[0])%MOD[0];
                h0 = (1LL*h0*pi[0][s])%MOD[0];

                long long h1 = (h[1][e]-h[1][s]+MOD[1])%MOD[1];
                h1 = (1LL*h1*pi[1][s])%MOD[1];

                return (h0<<32)|h1;
        }

        // SOLO PARA ROLLING / JOIN:
        long long join(long long a, long long b, int lenA){
                long long a0 = a>>32;
                long long a1 = a&0xffffffffLL;

                long long b0 = b>>32;
                long long b1 = b&0xffffffffLL;

                a0 = (a0 + b0*pw[0][lenA])%MOD[0];
                a1 = (a1 + b1*pw[1][lenA])%MOD[1];

                return (a0<<32)|a1;
        }
};
```

3.8. Hashing 2d

```cpp
/*
    2D Hashing: buscar todas las apariciones de una matriz
    r x c dentro de una matriz n x m.
*/

const ll BASE = 911382323LL;

void solve() {
    int r, c;
    cin >> r >> c;

    vector<string> pattern(r);
    for (int i = 0; i < r; i++)
        cin >> pattern[i];

    // Potencias para el rolling vertical.
    vector<ll> power(r + 1, 1);
    for (int i = 1; i <= r; i++)
        power[i] = power[i - 1] * BASE;

    // Hash de la matriz patr n.
    ll patternHash = 0;

    for (int i = 0; i < r; i++) {
        Hash h(pattern[i]);
        ll rowHash = h.get(0, c);

        patternHash += rowHash * power[r - i - 1];
    }

    int n, m;
    cin >> n >> m;

    vector<string> grid(n);
    for (int i = 0; i < n; i++)
        cin >> grid[i];

    // rowHash[i][j] = hash de grid[i][j ... j+c-1].
    vector<vector<ll>> rowHash(
        n, vector<ll>(m - c + 1)
    );

    for (int i = 0; i < n; i++) {
        Hash h(grid[i]);

        for (int j = 0; j <= m - c; j++)
            rowHash[i][j] = h.get(j, j + c);
    }

    // matrixHash[i][j] = hash de la submatriz r x c desde (i,j).
    vector<vector<ll>> matrixHash(
        n - r + 1,
        vector<ll>(m - c + 1)
    );

    for (int j = 0; j <= m - c; j++) {
        ll cur = 0;

        // Primera ventana vertical.
        for (int i = 0; i < r; i++)
            cur += rowHash[i][j] * power[r - i - 1];

        matrixHash[0][j] = cur;

        // Rolling vertical.
        for (int i = 1; i <= n - r; i++) {
            cur -= rowHash[i - 1][j] * power[r - 1];
            cur = cur * BASE + rowHash[i + r - 1][j];

            matrixHash[i][j] = cur;
        }
    }

    // 2D difference array para marcar las apariciones.
    vector<vector<int>> diff(
        n + 1, vector<int>(m + 1, 0)
    );

    for (int i = 0; i <= n - r; i++) {
        for (int j = 0; j <= m - c; j++) {

            if (matrixHash[i][j] == patternHash) {
                diff[i][j]++;
                diff[i][j + c]--;
                diff[i + r][j]--;
                diff[i + r][j + c]++;
```

```
            }
        }
    }

    // Prefijos 2D.
    for (int i = 0; i < n; i++)
        for (int j = 1; j < m; j++)
            diff[i][j] += diff[i][j - 1];

    for (int j = 0; j < m; j++)
        for (int i = 1; i < n; i++)
            diff[i][j] += diff[i - 1][j];

    // Imprimir celdas pertenecientes a alguna aparici n.
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            cout << (diff[i][j] > 0 ? grid[i][j] : '.');
        }
        cout << '\n';
    }
}
```

3.9. Manacher

```
const int N = 1e5 + 10;
int odd[N]; //odd[i] = max odd palindrome centered on i
int even[N]; //even[i] = max even palindrome centered on i
void manacher(string & s) {
    int l = 0, r = -1, n = s.size();
    for (int i = 0; i < n; i++) {
        int k = i > r ? 1 : min(odd[l + r - i], r - i);
        while (i + k < n and i - k >= 0 and s[i + k] == s[i - k]) k++;
        odd[i] = k--;
        if (i + k > r) l = i - k, r = i + k;
    }
    l = 0;
    r = -1;
    for (int i = 0; i < n; i++) {
        int k = i > r ? 0 : min(even[l + r - i + 1], r - i + 1);
        k++;
        while (i + k <= n and i - k >= 0 and s[i + k - 1] == s[i - k]) k++;
        even[i] = --k;
        if (i + k - 1 > r) l = i - k, r = i + k - 1;
    }
}
```

4. Bit Manipulation

4.1. Binary Representation

```
string tobit(long long x) {
    string a(64, '0');
    for (int i = 63; i >= 0; --i) {
        if (x & 1) a[i] = '1';
        x >>= 1;
    }
    return a;
}
```

4.2. Bit Mask

```
for(int mask = 0; mask < (1 << n); mask++) { //(1 << n) = O(2^n)
    for(int i = 0; i < n; i++) { // O(n)
    //verificamos si el i-esimo bit esta encendido
        if(mask & (1 << i)) {
            cout << v[i] << "␣";
        }
    }
}
```

4.3. Turn Off a Bit at Position X

```cpp
void solve(){
    //mapa de caracteres en windows para ~
    int a, x;
    cin >> a >> x; // a es el numero, x es el numero de bit
    //los bits se enumeran de dercha a izquierda iniciando de 0
    /*
    Ej: n = 8
    Binario:     1 0 0 0
    Posicion:    3 2 1 0
    */
    a = a & ~(1 << x); //apagamos el bit x de a
    cout << a << "\n";
}
```

4.4. Xor Basis

```cpp
struct LB {
    int b[30] = {}, sz = 0;
    bool zero = 0; // Indica si se puede formar el 0

    // 1. Inserta un n mero en la base
    void insert(int mask) {
        for (int i = 29; i >= 0; i--) {
            if ((mask >> i) & 1) {
                if (!b[i]) { b[i] = mask; sz++; return; }
                mask ^= b[i];
            }
        }
        zero = 1;
    }

    // 2. Comprueba si un valor espec fico se puede formar usando XOR
    bool query(int mask) {
        for (int i = 29; i >= 0; i--) {
            if ((mask >> i) & 1) {
                if (!b[i]) return 0;
                mask ^= b[i];
            }
        }
        return 1;
    }

    // 3. Maximiza un valor haciendo XOR con elementos de la base
    int maximize(int val) {
        for (int i = 29; i >= 0; i--) if ((val ^ b[i]) > val) val ^= b[i];
        return val;
    }

    // 4. Minimiza un valor haciendo XOR con elementos de la base
    int minimize(int val) {
        for (int i = 29; i >= 0; i--) if ((val ^ b[i]) < val) val ^= b[i];
        return val;
    }

    // 5. Retorna la cantidad total de valores  nicos  distintos (2^sz)
    long long count_distinct() {
        return 1LL << sz;
    }

    // 6. Obtiene el k- simo  valor m s peque o que se puede formar (0-indexed)
    int get_kth(long long k) {
        vector<int> active;
        for (int i = 0; i < 30; i++) if (b[i]) active.push_back(b[i]);
        if (k >= (1LL << active.size())) return -1;
        int res = 0;
        for (size_t i = 0; i < active.size(); i++) {
            if ((k >> i) & 1) res ^= active[i];
        }
        return res;
    }

    // 7. Fusiona otra base linear en esta
    void merge(const LB& other) {
        for (int i = 0; i < 30; i++) if (other.b[i]) insert(other.b[i]);
    }
};
```

5. Data Structures

5.1. Iterative Segment Tree

```cpp
struct info
{
    int num;
};

info ope(info u, info v){
    return {min(u.num, v.num)};
}

int sz;
vector<info> ST;

void build(int n) {
    sz = 1;
    while (sz < n) sz *= 2;
    ST.assign(2 * sz, {1000000000});
}

void update(int pos, int val) {
    pos += sz;
    ST[pos] = ope(ST[pos], {val});
    for (pos /= 2; pos > 0; pos /= 2) {
        ST[pos] = ope(ST[2 * pos], ST[2 * pos + 1]);
    }
}

info get(int l, int r) {
    info res = {1000000000};
    for (l += sz, r += sz + 1; l < r; l /= 2, r /= 2) {
        if (l & 1) res = ope(res, ST[l++]);
        if (r & 1) res = ope(res, ST[--r]);
    }
    return res;
}
//Las operaciones y consultas son iguales al recursivo, 0-index
```

5.2. Segment Tree

```cpp
const int N = 1e5;

struct info {
    int num;
};

info ST[4 * N];

info ope(info u, info v) {
    return {u.num + v.num};
}

void build(const vector<int> &v, int in = 1, int L = 0, int R = n - 1) {
    if (L == R) {
        ST[in] = {v[L]}; // Asignamos el valor desde el vector
    } else {
        int mid = (L + R) >> 1;
        build(v, 2 * in, L, mid);
        build(v, (2 * in) + 1, mid + 1, R);
        ST[in] = ope(ST[2 * in], ST[(2 * in) + 1]);
    }
}

void update(int pos, int val, int in = 1, int L = 0, int R = n - 1) {
    if (L == R) {
        ST[in].num = val;
    } else {
        int mid = (L + R) >> 1;
        if (pos <= mid) {
            update(pos, val, 2 * in, L, mid);
        } else {
            update(pos, val, (2 * in) + 1, mid + 1, R);
        }
        ST[in] = ope(ST[2 * in], ST[(2 * in) + 1]);
    }
}

info get(int l, int r, int in = 1, int L = 0, int R = n - 1) {
    if (r < L or R < l) return {0}; // Elemento neutro
```

```
    if (l <= L and R <= r) return ST[in];

    int mid = (L + R) >> 1;
    info left = get(l, r, 2 * in, L, mid);
    info right = get(l, r, (2 * in) + 1, mid + 1, R);
    return ope(left, right);
}


// Llamadas simples y limpias:
// build(v);              // Construye con el vector
// update(2, 10);         // Cambia el valor en pos 2 a 10
// info ans = get(1, 3); // Obtiene la respuesta en el rango [1, 3]
// v = {1 2 3 4 5}
// get(1, 0, n-1, 1, 3) = 2 + 3 + 4 = 9
```

5.3. Segment Tree Lazy Propagation

```
const int N = 1e5;

struct info {
    long long num;
};

info ST[4 * N];
long long lazy[4 * N];

info ope(info u, info v) {
    return {u.num + v.num};
}

void build(const vector<int> &v, int in = 1, int L = 0, int R = n - 1) {
    if (L == R) {
        ST[in] = {v[L]};
    } else {
        int mid = (L + R) >> 1;

        build(v, 2 * in, L, mid);
        build(v, 2 * in + 1, mid + 1, R);

        ST[in] = ope(ST[2 * in], ST[2 * in + 1]);
    }
}

void push(int in, int L, int R) {
    if (lazy[in] == 0)
        return;

    ST[in].num += lazy[in] * (R - L + 1);

    if (L != R) {
        lazy[2 * in] += lazy[in];
        lazy[2 * in + 1] += lazy[in];
    }

    lazy[in] = 0;
}

void update(int l, int r, long long val,
            int in = 1, int L = 0, int R = n - 1) {

    push(in, L, R);

    if (r < L or R < l)
        return;

    if (l <= L and R <= r) {
        lazy[in] += val;
        push(in, L, R);
        return;
    }

    int mid = (L + R) >> 1;

    update(l, r, val, 2 * in, L, mid);
    update(l, r, val, 2 * in + 1, mid + 1, R);

    ST[in] = ope(ST[2 * in], ST[2 * in + 1]);
}

info get(int l, int r,
         int in = 1, int L = 0, int R = n - 1) {

    push(in, L, R);
```

```

    if (r < L or R < l)
        return {0};

    if (l <= L and R <= r)
        return ST[in];

    int mid = (L + R) >> 1;

    info left = get(l, r, 2 * in, L, mid);
    info right = get(l, r, 2 * in + 1, mid + 1, R);

    return ope(left, right);
}
```

5.4. Segment Tree Lazy Struct

```
// CSES Range Update and Sums
// q = 1 sumar x a todo un rango
// q = 2 todos los valores en un rango deben ser igual a x

const int N = 2e5+10;

int n;

struct info{
    ll sum, estado, lazy;
};
info ST[4*N];

info ope(info a, info b){
    info aux;
    aux.lazy = 0;
    aux.estado = 0;
    aux.sum = a.sum + b.sum;
    return aux;
}

void build(const vector<ll> &v, int in = 1, int L = 0, int R = n-1){
    if(L == R){
        ST[in] = {v[L], 0 , 0};
        return;
    }
    int mid = (R+L)/2;
    build(v, 2*in, L, mid);
    build(v, 2*in+1, mid+1, R);
    ST[in] = ope (ST[2*in], ST[2*in+1]);;
}

void push(int in, int L, int R){

    if(ST[in].estado == 0){
        return;
    }

    if(ST[in].estado == 1){
        ST[in].sum += ST[in].lazy * (R - L + 1);
        if(L != R){
            ST[2*in].lazy += ST[in].lazy;
            ST[2*in+1].lazy += ST[in].lazy;
            if(ST[2*in].estado == 0){
                ST[2*in].estado = ST[in].estado;
            }
            if(ST[2*in+1].estado == 0){
                ST[2*in+1].estado = ST[in].estado;
            }
        }
    }
    else{
        ST[in].sum = ST[in].lazy * (R - L + 1);
        if(L != R){
            ST[2*in].lazy = ST[in].lazy;
            ST[2*in+1].lazy = ST[in].lazy;
            ST[2*in].estado = ST[in].estado;
            ST[2*in+1].estado = ST[in].estado;
        }
    }
    ST[in].estado = 0;
    ST[in].lazy = 0;
}

void update(int l, int r, ll val, ll estado, int in = 1, int L = 0, int R = n-1){
    push(in, L, R);
    if(r < L or R < l){
```

```
        return ;
    }
    if(l <= L and R <= r){
        ST[in].estado = estado;
        if(estado == 1){
            ST[in].lazy += val;
            push(in, L, R);
        }
        else{
            ST[in].lazy = val;
            push(in, L, R);
        }
        return;
    }

    int mid = (R+L)/2;
    update(l, r, val, estado, 2*in, L, mid);
    update(l, r, val, estado, 2*in+1, mid+1, R);
    ST[in] = ope(ST[2*in], ST[2*in+1]);
}

info query(int l, int r, int in = 1, int L = 0, int R = n-1){
    push(in, L, R);
    if(r < L or R < l){
        return {0, 0 ,0};
    }

    if(l <= L and R <= r){
        return ST[in];
    }

    int mid = (R+L)/2;
    info left = query(l, r, 2*in, L, mid);
    info right = query(l, r, 2*in+1, mid+1, R);
    return ope(left, right);
}


void solve(){
    int Q;
    cin >> n >> Q;
    vector<ll>v(n);
    repl(i,0,n){
        cin >> v[i];
    }
    build(v);
    while(Q--){
        int q;
        cin >> q;
        // dbg(q);
        if(q==1 or q==2){
            int l, r;
            cin >> l >> r;
            ll x;
            l--, r--;
            cin >> x;
            update(l, r, x, q);
        }
        else{
            int l, r;
            cin >> l >> r;
            l--, r--;
            ll ans = query(l, r).sum;
            cout << ans << "\n";
        }
    }

}
```

5.5. 1D Binary Indexed Tree

```
struct BIT{
    int n;
    vector<ll> bit;
    BIT(int _n){
        n = _n;
        bit.assign(n + 1, 0);
    }
    void add(int x, ll val){
        for (int i = x; i <= n; i += (i&(-i))){
            bit[i] += val;
        }

    }
```

```
    ll sum(int x){
        ll ans = 0ll;
         for (int i = x; i > 0; i -= (i&(-i))){
            ans += bit[i];
        }
        return ans;
    }
    ll query(int l, int r){
        return sum(r) - sum(l - 1);
    }
};
// Para actulizar en rango y consultas puntuales
// bit.add(l,val); bit.add(r+1,val);
// bit.query(1,pos)
```

5.6. 2D Binary Indexed Tree

```
struct BIT{
    int n, m;
    vector<vector<ll>> bit;

    BIT(int _n){
        n = _n;
        m = _n;
        bit.assign(n+1, vector<ll>(n+1 ,0));
    }
    void add(int x, int y, ll val){
        for (int i = x; i <= n; i += (i&(-i))){
            for (int j = y; j <= n; j += (j&(-j))){
                bit[i][j] += val;
            }
        }
    }
    ll sum(int x, int y){
        ll ans = 0ll;
        for (int i = x; i > 0; i -= (i&(-i))){
            for (int j = y; j > 0; j -= (j&(-j))){
                ans += bit[i][j];
            }
        }
        return ans;
    }
    ll query(int x1, int y1, int x2, int y2){
        return sum(x2, y2) - sum(x2, y1 - 1) - sum(x1 - 1, y2) + sum(x1 - 1, y1 - 1);
    }
};
```

5.7. 2D Binary Indexed Tree Infinite Plane

```
struct BIT{
    ll n;
    vector<vector<ll>> xs;
    vector<vector<ll>> bit;
    BIT(ll nn){
        n = nn;
        xs.resize(n + 1);
        bit.resize(n + 1);
    }
    void prepare(ll x, ll y){
        for (ll i = x; i <= n; i += (i&(-i))){
            xs[i].push_back(y);
        }
    }
    void build(){
        for (ll i = 1; i <= n; i++){
            sort(xs[i].begin(), xs[i].end());
            xs[i].erase(unique(xs[i].begin(), xs[i].end()), xs[i].end());
            bit[i].assign(xs[i].size() + 1, 0);
        }
    }
    void add(ll x, ll y, ll val){
        for (ll i = x; i <= n; i += (i&(-i))){
            ll p = lower_bound(xs[i].begin(), xs[i].end(), y) - xs[i].begin() + 1;
            for (ll j = p; j < bit[i].size(); j += (j&(-j))){
                bit[i][j] += val;
            }
        }
    }
    ll sum(ll x, ll y){
        ll ans = 0ll;
        for (ll i = x; i > 0; i -= (i&(-i))){
```

```
            ll p = lower_bound(xs[i].begin(), xs[i].end(), y) - xs[i].begin();
            for (ll j = p; j > 0; j -= (j&(-j))){
                ans += bit[i][j];
            }
        }
        return ans;
    }
    ll query(ll x1, ll y1, ll x2, ll y2){
        return sum(x2-1,y2) - sum(x2-1, y1) - sum(x1 - 1, y2) + sum(x1 - 1, y1);
    }

};
void solve(int tc){
    int n,q; cin >> n >> q;
    set<ll> X;
    set<ll> Y;
    vector<vector<ll>> points;
    vector<vector<ll>> query;
    repl(i,0,n){
        ll x,y,w; cin >> x >> y >> w;
        X.insert(x);
        Y.insert(y);
        points.push_back({x,y,w});
    }
    repl(i,0,q){
        int t; cin >> t;
        if(t == 0){
            ll x,y,w; cin >> x >> y >> w;
            X.insert(x);
            Y.insert(y);
            query.push_back({0,x,y,w});
        }
        else{
            ll x1, y1, x2, y2;
            cin >> x1 >> y1 >> x2 >> y2;
            X.insert(x1);
            X.insert(x2);
            Y.insert(y1);
            Y.insert(y2);
            query.push_back({1,x1,y1,x2,y2});
        }
    }
    map<ll,ll> equis, ye;
    int i = 0;
    for(ll x : X){
        i++;
        equis[x] = i;
    }
    i = 0;
    for(ll x : Y){
        i++;
        ye[x] = i;
    }
    BIT bit((ll)X.size());
    for(auto &x : points){
        x[0] = equis[x[0]];
        x[1] = ye[x[1]];
        bit.prepare(x[0], x[1]);
    }
    for(auto &x : query){
        if(x[0] == 0){
            x[1] = equis[x[1]];
            x[2] = ye[x[2]];
            bit.prepare(x[1], x[2]);
        }
        else{
            x[1] = equis[x[1]];
            x[2] = ye[x[2]];
            x[3] = equis[x[3]];
            x[4] = ye[x[4]];
        }
    }
    bit.build();
    for(auto &x : points){
        bit.add(x[0], x[1], x[2]);
    }
    for(auto &x : query){
        if(x[0] == 0){
            bit.add(x[1], x[2], x[3]);
        }
        else{
            cout << bit.query(x[1],x[2],x[3],x[4]) << "\n";
        }
    }
}
```

5.8. 3D Binary Indexed Tree

```
struct BIT {
    int n;
    int m;
    int p;
    vector < vector < vector < ll >>> bit;
    BIT(int _n) {
        n = _n + 10;
        m = _n + 10;
        p = _n + 10;
        bit.assign(n, vector < vector < ll >> (n, vector < ll > (n, 0)));
    }
    void add(int x, int y, int z, int val) {
        for (int i = x; i <= n; i += (i & (-i))) {
            for (int j = y; j <= n; j += (j & (-j))) {
                for (int k = z; k <= n; k += (k & (-k))) {
                    bit[i][j][k] += val;
                }
            }
        }
    }
    ll sum(int x, int y, int z) {
        ll ans = 0 ll;
        for (int i = x; i > 0; i -= (i & (-i))) {
            for (int j = y; j > 0; j -= (j & (-j))) {
                for (int k = z; k > 0; k -= (k & (-k))) {
                    ans += bit[i][j][k];
                }
            }
        }
        return ans;
    }
    ll query(int x1, int y1, int z1, int x2, int y2, int z2) {
        return sum(x2, y2, z2) - sum(x1 - 1, y2, z2) - sum(x2, y2, z1 - 1) - sum(x2, y1 - 1, z2
            ) + sum(x1 - 1, y1 - 1, z2) + sum(x2, y1 - 1, z1 - 1) + sum(x1 - 1, y2, z1 - 1) -
            sum(x1 - 1, y1 - 1, z1 - 1);
    }
};
```

5.9. Disjoint Set Union

```
struct DSU{
    int num;
    vector<int> parent;
    vector<int> sizes;
    DSU(){};
    DSU(int n){
        init(n);
    }
    void init(int n){
        num = n;
        parent.resize(n+1);
        sizes.resize(n+1);
        //si tienes index 0 inicializar desde 0
        for(int i = 1; i <= n ; i++){
            parent[i] = i;
            sizes[i] = 1;
        }
    }
    int find(int a){
        if(a == parent[a]) return a;
        return parent[a] = find(parent[a]);
    }
    void join(int a, int b){
        int x = find(a);
        int y = find(b);
        if(x == y) return;
        num--;
        if(sizes[x] < sizes[y]){
            parent[x] = y;
            sizes[y] += sizes[x];
        }
        else{
            parent[y] = x;
            sizes[x] += sizes[y];
        }

    }
};
//Sabrossus
```

5.10. Dsu Dynamic Representative

```
int n, q;
cin >> n >> q;
DSU dsu(n + q + 10);
vector<int> id(n + 1),box(n + q + 10);
int nodes = n;
iota(id.begin(), id.end(), 0);
iota(box.begin(), box.end(), 0);
for(int i = 0 ;  i < q ; i ++){
    int a, b;
    cin >> a >> b;
    int ori = id[a];
    int ori2 = id[b];
    dsu.parent[ori] = ori2;
    nodes++;
    id[a] = nodes;
    box[nodes] = a;
}
for(int i = 1; i  <= n  ; i ++){
    cout << box[dsu.find(i)] << ' ';
}
```

5.11. Dynamic Connectivity Offline

```
struct Change {
    int a, b, sz_a;
};

struct DSU {
    vector<int> parent, sz;
    vector<Change> history;
    int components;

    DSU(int n) {
        parent.resize(n + 1);
        sz.assign(n + 1, 1);
        iota(parent.begin(), parent.end(), 0);
        components = n;
    }

    int find(int x) {
        while (x != parent[x])
            x = parent[x];
        return x;
    }

    bool unite(int a, int b) {
        a = find(a);
        b = find(b);
        if (a == b) {
            history.push_back({-1, -1, -1});
            return false;
        }
        if (sz[a] < sz[b])
            swap(a, b);
        history.push_back({a, b, sz[a]});
        parent[b] = a;
        sz[a] += sz[b];
        components--;
        return true;
    }

    int snapshot() {
        return history.size();
    }

    void rollback(int snap) {
        while (history.size() > snap) {
            auto [a, b, old_sz_a] = history.back();
            history.pop_back();
            if (a == -1)
                continue;
            parent[b] = b;
            sz[a] = old_sz_a;
            components++;
        }
    }
};

struct Edge {
    int u, v;
};
```

```

struct SegmentTree {
    int n;
    vector<vector<Edge>> tree;
    SegmentTree(int n) {
        this->n = n;
        tree.resize(4 * n + 5);
    }
    void add(int node, int l, int r, int ql, int qr, Edge e) {
        if (qr < l or r < ql)
            return;
        if (ql <= l and r <= qr) {
            tree[node].push_back(e);
            return;
        }
        int mid = (l + r) / 2;
        add(node * 2, l, mid, ql, qr, e);
        add(node * 2 + 1, mid + 1, r, ql, qr, e);
    }
    void add(int l, int r, Edge e) {
        if (l > r)
            return;
        add(1, 1, n, l, r, e);
    }
};

void dfs(int node, int l, int r, SegmentTree &seg, DSU &dsu, vector<int> &answer){
    int snap = dsu.snapshot();
    for (auto [u, v] : seg.tree[node])
        dsu.unite(u, v);
    if (l == r) {
        answer[l] = dsu.components;
    } else {
        int mid = (l + r) / 2;
        dfs(node * 2, l, mid, seg, dsu, answer);
        dfs(node * 2 + 1, mid + 1, r, seg, dsu, answer);
    }
    dsu.rollback(snap);
}

void solve() {
    int n, q;
    cin >> n >> q;
    if (q == 0)
        return;
    DSU dsu(n);
    SegmentTree seg(q);
    map<pair<int,int>, int> start;
    vector<int> answer(q + 1);
    vector<bool> query(q + 1);
    for (int t = 1; t <= q; t++) {
        char type;
        cin >> type;
        if (type == '?') {
            query[t] = true;
        } else {
            int u, v;
            cin >> u >> v;
            if (u > v)
                swap(u, v);
            if (type == '+') {
                start[{u, v}] = t;
            } else {
                int L = start[{u, v}];
                seg.add(L, t - 1, {u, v});
                start.erase({u, v});
            }
        }
    }
    for (auto [edge, L] : start) {
        auto [u, v] = edge;
        seg.add(L, q, {u, v});
    }
    dfs(1, 1, q, seg, dsu, answer);
    for (int t = 1; t <= q; t++) {
        if (query[t])
            cout << answer[t] << '\n';
    }
}
```

5.12. Implicit Treap

```
// Mantiene las posiciones
struct Node {
```

```
    int val , pri , sz ;
    Node *l , *r;
    Node ( int v) {
        val = v;
        pri = rand () ;
        sz = 1;
        l = r = nullptr ;
    }
};
using pnode = Node *;
int sz ( pnode t) {
    return t ? t -> sz : 0;
}
void upd (pnode t) {
    if (t) t -> sz = 1 + sz (t -> l) + sz (t -> r);
}
void split ( pnode t , pnode &l , pnode &r , int k) {
    if (! t) {
        l = r = nullptr ;
        return ;
    }
    int leftSize = sz (t -> l);
    if ( leftSize >= k) {
        split (t -> l , l , t ->l , k) ;
        r = t;
    }
    else {
        split (t -> r , t ->r , r , k - leftSize - 1) ;
        l = t;
    }
    upd (t);
}
pnode merge ( pnode l , pnode r) {
    if (! l || !r ) return l ? l : r;

    if (l -> pri > r -> pri ) {
        l -> r = merge (l -> r , r);
        upd (l );
        return l;
    }
    else {
        r -> l = merge (l , r ->l );
        upd (r );
        return r;
    }
}
void insert ( pnode & root , int pos , int val ) {
    pnode a , b;
    split ( root , a , b , pos );
    root = merge ( merge (a , new Node ( val )) , b) ;
}
void erase ( pnode & root , int pos ) {
    pnode a , b , c;
    split ( root , a , b , pos );
    split (b , b , c , 1) ;
    root = merge (a , c) ;
}
void print(pnode t) {
    if (!t) return;

    print(t->l);
    cout << t->val << "␣";
    print(t->r);
}
int get(pnode t, int k) {
    if (!t) return -1;

    int leftSize = sz(t->l);

    if (k < leftSize)
        return get(t->l, k);

    if (k == leftSize)
        return t->val;

    return get(t->r, k - leftSize - 1);
}
```

5.13. Mergesort Tree

```
#include <bits/stdc++.h>
using namespace std;

// ============================================================
```

```cpp
// Merge Sort Tree
// Cada nodo guarda el segmento [l, r] del arreglo ORDENADO.
//
// Construccion: O(n log n) tiempo y memoria
// Consultas   : O(log^2 n)   (log n nodos * busqueda binaria)
//
// Sirve para (sin updates):
//   - cuantos elementos < x (o <= x) en [l, r]
//   - cuantos elementos en rango de valores [lo, hi] dentro de [l, r]
//   - k-esimo menor en [l, r]  (busqueda binaria sobre el valor)
//
// Indices 0-based, consultas con [l, r] inclusivo.
// ============================================================
struct MergeSortTree {
    int n;
    vector<vector<int>> t;  // t[nodo] = elementos del segmento, ordenados

    MergeSortTree() {}
    MergeSortTree(const vector<int>& a) { build(a); }

    void build(const vector<int>& a) {
        n = a.size();
        t.assign(4 * n, {});
        build(1, 0, n - 1, a);
    }

    void build(int node, int l, int r, const vector<int>& a) {
        if (l == r) {                  // hoja: un solo elemento
            t[node] = {a[l]};
            return;
        }
        int m = (l + r) / 2;
        build(2 * node, l, m, a);
        build(2 * node + 1, m + 1, r, a);
        // mezclar los dos hijos ya ordenados (como en merge sort)
        t[node].resize(r - l + 1);
        merge(t[2 * node].begin(), t[2 * node].end(),
              t[2 * node + 1].begin(), t[2 * node + 1].end(),
              t[node].begin());
    }

    // cantidad de elementos < x en [ql, qr]
    int count_less(int ql, int qr, int x) { return count_less(1, 0, n - 1, ql, qr, x); }

    int count_less(int node, int l, int r, int ql, int qr, int x) {
        if (qr < l || r < ql) return 0;          // fuera del rango
        if (ql <= l && r <= qr)                  // totalmente dentro: busqueda binaria
            return lower_bound(t[node].begin(), t[node].end(), x) - t[node].begin();
        int m = (l + r) / 2;
        return count_less(2 * node, l, m, ql, qr, x) +
               count_less(2 * node + 1, m + 1, r, ql, qr, x);
    }

    // cantidad de elementos <= x en [ql, qr]
    int count_leq(int ql, int qr, int x) { return count_less(ql, qr, x + 1); }  // solo enteros

    // cantidad de elementos con valor en [lo, hi] dentro de [ql, qr]
    int count_range(int ql, int qr, int lo, int hi) {
        return count_leq(ql, qr, hi) - count_less(ql, qr, lo);
    }

    // k-esimo menor (k = 1 es el minimo) en [ql, qr]
    // busqueda binaria sobre el valor: O(log(MAXV) * log^2 n)
    int kth(int ql, int qr, int k, int lo = -1e9, int hi = 1e9) {
        while (lo < hi) {
            int mid = lo + (hi - lo) / 2;        // piso seguro para negativos
            if (count_leq(ql, qr, mid) >= k) hi = mid;
            else lo = mid + 1;
        }
        return lo;
    }
};

// ---------------- Ejemplo de uso ----------------
// int main() {
//     vector<int> a = {5, 1, 4, 2, 3};
//     MergeSortTree mst(a);
//     cout << mst.count_less(0, 4, 4) << "\n";        // 3  (1, 2, 3)
//     cout << mst.count_range(1, 3, 2, 4) << "\n";    // 2  (4, 2)
//     cout << mst.kth(0, 4, 2) << "\n";               // 2
// }
```

5.14. Ordered Set

```cpp
#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>

using namespace std;
using namespace __gnu_pbds;

typedef tree<int, null_type, less<int>, rb_tree_tag, tree_order_statistics_node_update>
    ordered_set;

void solve() {
    ordered_set st;

    st.insert(10);
    st.insert(30);
    st.insert(20);

    // 1. Elemento en la posicion K (0-indexed, de menor a mayor)
    // find_by_order devuelve un iterador (puntero). Usamos '*' para el valor.
    auto it = st.find_by_order(1);
    if (it != st.end()) {
        int elemento = *it; // Devuelve 20 de forma segura
    }

    // 2. Cuantos elementos son estrictamente menores que X
    int menores_que_25 = st.order_of_key(25); // Devuelve 2 (el 10 y el 20)
}
// Sabrossus
```

5.15. Range Bit

```cpp
struct RangeBIT{
    int n;
    vector<ll> bit1, bit2;
    RangeBIT(int nn){
        n = nn;
        bit1.assign(n + 1,0);
        bit2.assign(n + 1,0);
    }
    void add_one(vector<ll> &b, int x, ll val){
        for (int i = x; i <= n; i+= (i&(-i))){
            b[i] += val;
        }
    }
    void add(int l, int r, ll val){
        add_one(bit1, l, val);
        add_one(bit1, r + 1, -val);
        add_one(bit2, l, l * val);
        add_one(bit2, r + 1, -(r + 1) * val);
    }
    ll sum_one(const vector<ll> &b, int x){
        ll ans = 0ll;
        for (int i = x; i > 0; i -= (i&(-i))){
            ans += b[i];
        }
        return ans;
    }

    ll sum(int x){
        return (x + 1) * sum_one(bit1,x) - sum_one(bit2, x);
    }
    ll query(int l, int r){
        return sum(r) - sum(l - 1);
    }
};
```

5.16. Sparse Table

```cpp
const int N = 1e5;
const int LOG = 20;
int v[N]; //values
int st[N][LOG]; //Sparse Table
int lg[N]; // log values
int n;
void build(){
    for(int i = 0; i < n; i++) st[i][0] = v[i];

    for(int i = 1; i < LOG; i++){
        for(int j = 0; j < n - (1<<i) + 1; j++){
            st[j][i] = min(st[j][i-1],st[j+(1<<(i-1))][i-1]);
```

```
            }
        }
        lg[1] = 0;
        for(int i = 2; i <= n;i++) lg[i] = lg[i/2] + 1;
    }
    int query(int L, int R){
        int k = lg[R-L+1];
        return min(st[L][k],st[R-(1<<k)+1][k]);
    }
    // v = [5, 2, 4, 7, 1, 3]
    // query(1, 4)     [2, 4, 7, 1]
```

5.17. Treap

```
// Ordena los elementos por valor
struct Node {
    int key, pri, sz;
    Node *l, *r;
    Node(int k) {
        key = k;
        pri = rand();
        sz = 1;
        l = r = nullptr;
    }
};

using pnode = Node*;

int sz(pnode t) {
    return t ? t->sz : 0;
}

void upd(pnode t) {
    if (t) t->sz = 1 + sz(t->l) + sz(t->r);
}

void split(pnode t, pnode &l, pnode &r, int k) {
    if (!t) {
        l = r = nullptr;
        return;
    }

    int leftSize = sz(t->l);

    if (leftSize >= k) {
        split(t->l, l, t->l, k);
        r = t;
    } else {
        split(t->r, t->r, r, k - leftSize - 1);
        l = t;
    }

    upd(t);
}

pnode merge(pnode l, pnode r) {
    if (!l || !r) return l ? l : r;

    if (l->pri > r->pri) {
        l->r = merge(l->r, r);
        upd(l);
        return l;
    } else {
        r->l = merge(l, r->l);
        upd(r);
        return r;
    }
}

void insert(pnode &root, int pos, int val) {
    pnode a, b;
    split(root, a, b, pos);
    root = merge(merge(a, new Node(val)), b);
}

void erase(pnode &root, int pos) {
    pnode a, b, c;
    split(root, a, b, pos);
    split(b, b, c, 1);
    root = merge(a, c);
}

int get(pnode t, int k) {
    if (!t) return -1;
```

```

    int leftSize = sz(t->l);

    if (k < leftSize)
        return get(t->l, k);

    if (k == leftSize)
        return t->key;

    return get(t->r, k - leftSize - 1);
}

void print(pnode t) {
    if (!t) return;

    print(t->l);
    cout << t->key << " ";
    print(t->r);
}
```

6. Dynamic Programing

6.1. Bounded Knapsack

```
// bounded_knapsack para DPs del estilo :
// dp[j - cw] + cv , w-> pesov-> valor y quieres maximizar el valor
int n, we;
cin >> n >> we;
vector<int> v, m, w, dp;
v.resize(n);
m.resize(n);
w.resize(n);
dp.assign(we+1, 0);
int sum = 0;
for(int i  =0 ; i  < n ; i++){
    int a, b, c;
    cin >> a >> b >> c;
    v[i] = a;
    w[i] = b;
    m[i] = c;
    vector<int> old = dp;
    for(int r = 0; r <w[i] and r <= we ;r++){
        deque<pair<int,int>> dq;
        for(int j = r; j <= we ;j +=w[i]){
            int k = (j - r)/w[i];
            while(!dq.empty() and dq.front().first < k - m[i]){
                dq.pop_front();
            }
            int cur = old[j] - k * v[i];
            while(!dq.empty() && dq.back().second <= cur){
                dq.pop_back();
            }
            dq.push_back({k,cur});
            dp[j] = dq.front().second + k * v[i];
        }
    }
}
cout << dp[we];
```

6.2. Digit Dp

```
//Counting Numbers CSES
//PyroxBoy
ll dp[20][11][2][2];
//pos, last, tight, started for leading zeroes

ll a, b;

int tam;
string aux;

ll f(int pos, int last, int tight, int start){
    if(pos > tam){
        return 1;
    }
    ll &ans = dp[pos][last][tight][start];
    if(ans == -1){
        ans = 0;
```

```
        int lim = 9;

        if(tight){
            lim = aux[pos] - '0';
        }
        int ultimo = aux[pos] - '0';
        for(int d = 0; d <= lim; d++){
            if(d == last and start){
                continue;
            }
            int ntight = 0;
            if(tight and d == ultimo){
                ntight = 1;
            }
            int nstart = start;
            if(!start and d != 0){
                nstart = 1;
            }
            ans += f(pos+1, d, ntight, nstart);
        }
    }
    return ans;
}

ll reco(ll x){
    if(x < 0){
        return 0;
    }
    aux = to_string(x);
    memset(dp, -1, sizeof(dp));
    tam = aux.size()-1;
    return f(0, 10, 1, 0);
}
void solve(){
    cin >> a >> b;
    f(0, 10, 1, 0);
    ll ans = reco(b) - reco(a-1);
    cout << ans << "\n";
}

//Sabroseando el code.
//Leer es clave.
//Upsolvear duele.
//Perseverancia, Paciencia, Providencia.
//Diamonds are made under pressure
```

6.3. Fast Fibonacci

```
#include <bits/stdc++.h>
#define ll long long

using namespace std;

const ll MOD = 1e9 + 7;

struct Matrix {
    ll a[2][2];
    Matrix() {
        memset(a, 0, sizeof(a));
    }
};

Matrix multiply(Matrix A, Matrix B) {
    Matrix C;
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++) {
            for (int k = 0; k < 2; k++) {
                C.a[i][j] = (C.a[i][j] + A.a[i][k] * B.a[k][j]) % MOD;
            }
        }
    }
    return C;
}

Matrix power(Matrix A, ll n) {
    Matrix R;
    R.a[0][0] = 1;
    R.a[1][1] = 1;
    while (n) {
        if (n & 1)
            R = multiply(R, A);
        A = multiply(A, A);
        n >>= 1;
    }
```

```
    return R;
}

ll fibonacci(ll n) {
    if (n == 0) return 0;
    Matrix A;
    A.a[0][0] = 1;
    A.a[0][1] = 1;
    A.a[1][0] = 1;
    A.a[1][1] = 0;
    Matrix R = power(A, n - 1);
    return R.a[0][0];
}

int main() {
    ll n;
    cin >> n;
    cout << fibonacci(n) << '\n';
}
```

6.4. Kadane 2d

```
int n;
cin >> n;
vector < vector < int >> v(n);
repl(i, 0, n) {
    repl(j, 0, n) {
        int x;
        cin >> x;
        v[i].push_back(x);
    }
}
int maxi = INT_MIN;
repl(ini, 0, n) {
    vector < int > pref(n, 0);
    repl(i, ini, n) {
        repl(j, 0, n) {
            pref[j] += v[i][j];
        }
        int cur = 0;
        repl(j, 0, n) {
            cur = max(pref[j], pref[j] + cur);
            maxi = max(maxi, cur);
        }
    }
}
```

6.5. Kadane 3d

```
int n;
cin >> n;

repl(i, 0, n) {
    repl(j, 0, n) {
        repl(k, 0, n) {
            cin >> v[i][j][k];
        }
    }
}
int maxi = INT_MIN;
repl(k_ini, 0, n) {
    vector < vector < int >> pref2(n, vector < int > (n, 0));
    repl(k, k_ini, n) {
        repl(i, 0, n) {
            repl(j, 0, n) {
                pref2[i][j] += v[i][j][k];
            }
        }
        repl(ini, 0, n) {
            vector < int > pref(n, 0);
            repl(i, ini, n) {
                repl(j, 0, n) {
                    pref[j] += pref2[i][j];
                }
                int cur = 0;
                repl(u, 0, n) {
                    cur = max(pref[u], cur + pref[u]);
                    maxi = max(maxi, cur);
                }
            }

```

```
            }
        }
    }

    cout << maxi << "\n";
```

6.6. Longest Increasing Subsequence

```
vector<ll> get_lis(vector<ll> &v) {
    vector<ll> ans;

    for (ll x : v) {
        if (ans.empty() || x > ans.back()) {
            ans.push_back(x);
        } else {
            ll low = lower_bound(ans.begin(), ans.end(), x) - ans.begin();
            ans[low] = x;
        }
    }

    return ans;
}
```

6.7. Matrix Exponentiation

```
ll const MOD = 1e9+7;

struct Mat {
    int n;
    vector<vector<unsigned int>> a;

    Mat(int n, bool id = false) : n(n), a(n, vector<unsigned int>(n, 0)) {
        if (id) for (int i = 0; i < n; i++) a[i][i] = 1;
    }

    Mat operator*(const Mat& o) const {
        Mat r(n);
        for (int i = 0; i < n; i++)
            for (int k = 0; k < n; k++) {
                if (a[i][k] == 0) continue;
                for (int j = 0; j < n; j++)
                    r.a[i][j] = (r.a[i][j] + a[i][k] * o.a[k][j])%MOD;
            }
        return r;
    }
};

Mat mpow(Mat b, ll e) {
    Mat r(b.n, true);
    while (e) {
        if (e & 1) r = r * b;
        b = b * b;
        e >>= 1;
    }
    return r;
}
```

6.8. Sos Dp

```
void sos_dp(vector<ll>& dp, int K){
//  k es el tamanho del arreglo :v
    for(int bit = 0; bit < K; bit++){
        for(int mask = 0; mask < (1 << K); mask++){
            if(mask & (1 << bit)){
                dp[mask] += dp[mask ^ (1 << bit)];
//              si quieres que sea un sos dp de maximos
//              dp[mask] = max(dp[mask], dp[mask ^ (1 << bit)]);
            }
        }
    }
}
```

6.9. Knapsack with Bitset

```cpp
// Uso: saber que sumas se pueden formar eligiendo un subconjunto.
// Costo: O(n * M / 64), M = suma maxima. Memoria: M/8 bytes.
const int M = 1000001;          // tama o FIJO en compilacion (suma max + 1)
static bitset<M> S;
S[0] = 1;
for (int v : vals) S |= S << v; // no tomo v | si tomo v

// Operaciones utiles
S.count();                      // cuantas sumas distintas
S[k];                           // se puede formar k?
S._Find_first(); S._Find_next(i); // recorrer sumas alcanzables
(S << x)                        // sumar x a TODOS los elementos
A | B                           // union de conjuntos
```

7. Fft

7.1. Ntt

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
typedef vector<ll> poly;

//  NTT (Number Theoretic Transform): FFT con enteros modulo primo
// Requisitos: MOD primo y (MOD-1) multiplo del tamanho (potencia de 2).
// Primos utiles (todos con raiz primitiva 3):
//   998244353  = 119 * 2^23 + 1  -> tamanho maximo 2^23
//   469762049  =   7 * 2^26 + 1  -> tamanho maximo 2^26
//   167772161  =   5 * 2^25 + 1  -> tamanho maximo 2^25
// Si el problema pide otro modulo (ej. 1e9+7): usar multiplicar_mod de fft
const ll MOD = 998244353, RT = 3;

ll addmod(ll a, ll b, ll mod = MOD) { a += b; if (a >= mod) a -= mod; return a; }
ll submod(ll a, ll b, ll mod = MOD) { a -= b; if (a < 0) a += mod; return a; }
ll mulmod(ll a, ll b, ll mod = MOD) { return a * b % mod; }

// b^e mod mod. O(log e)
ll pot(ll b, ll e, ll mod = MOD) {
        ll res = 1;
        b %= mod;
        while (e) {
                if (e & 1) res = res * b % mod;
                b = b * b % mod;
                e >>= 1;
        }
        return res;
}

// Inverso modular (Fermat, mod primo)
ll inv(ll a, ll mod = MOD) { return pot(a, mod - 2, mod); }

// Transformada in-place. a.size() debe ser potencia de 2.
// invertir = false: coeficientes -> valores
// invertir = true : valores -> coeficientes
// O(n log n)
void ntt(poly& a, bool invertir, ll mod = MOD, ll rt = RT) {
        int n = a.size();
        // 1) Permutacion bit-reversal
        for (int i = 1, j = 0; i < n; i++) {
                int bit = n >> 1;
                for (; j & bit; bit >>= 1) j ^= bit;
                j ^= bit;
                if (i < j) swap(a[i], a[j]);
        }
        // 2) Mariposas
        for (int len = 2; len <= n; len <<= 1) {
                ll wl = pot(rt, (mod - 1) / len, mod); // raiz primitiva len-esima de la unidad
                if (invertir) wl = inv(wl, mod);
                for (int i = 0; i < n; i += len) {
                        ll w = 1;
                        for (int j = 0; j < len / 2; j++) {
                                ll u = a[i + j];
                                ll v = a[i + j + len / 2] * w % mod;
                                a[i + j] = addmod(u, v, mod);
                                a[i + j + len / 2] = submod(u, v, mod);
                                w = w * wl % mod;
                        }
                }
        }
```

```
        // 3) La inversa divide entre n
        if (invertir) {
                ll ninv = inv(n, mod);
                for (auto& x : a) x = x * ninv % mod;
        }
}

// Producto de polinomios modulo mod. Resultado de tamanho a.size()+b.size()-1.
// Los coeficientes de entrada deben estar en [0, mod). O((n+m) log(n+m))
poly multiply(const poly& a, const poly& b, ll mod = MOD, ll rt = RT) {
        if (a.empty() || b.empty()) return {};
        int need = a.size() + b.size() - 1;
        int n = 1;
        while (n < need) n <<= 1;
        poly fa(a), fb(b);
        fa.resize(n);
        fb.resize(n);
        ntt(fa, false, mod, rt);
        ntt(fb, false, mod, rt);
        for (int i = 0; i < n; i++) fa[i] = fa[i] * fb[i] % mod;
        ntt(fa, true, mod, rt);
        fa.resize(need);
        return fa;
}

// ============================================================
//  Producto EXACTO (sin modulo) con dos NTT + CRT
// ============================================================
// Sirve cuando los coeficientes reales del resultado son < ~4.6e17
// (m1*m2) y los de entrada son >= 0. Por ejemplo, resultados ~1e10 que
// no caben en un solo modulo.
// CRT: x = r1 + m1 * ((r2 - r1) * inv(m1) mod m2)
poly multiplicar_crt(const poly& a, const poly& b) {
        const ll m1 = 998244353, m2 = 469762049;
        poly a1(a), b1(b), a2(a), b2(b);
        for (auto& x : a1) x %= m1;
        for (auto& x : b1) x %= m1;
        for (auto& x : a2) x %= m2;
        for (auto& x : b2) x %= m2;
        poly r1 = multiply(a1, b1, m1, 3);
        poly r2 = multiply(a2, b2, m2, 3);
        ll inv_m1 = inv(m1 % m2, m2);
        poly res(r1.size());
        for (int i = 0; i < (int)res.size(); i++) {
                ll k = submod(r2[i], r1[i] % m2, m2) * inv_m1 % m2;
                res[i] = r1[i] + m1 * k; // < m1*m2 ~ 4.6e17, cabe en ll
        }
        return res;
}
```

7.2. Fft

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
typedef complex<double> cd;

//  FFT con numeros complejos (double)
// Cuando usarla: resultados enteros "grandes" que NO se quieren
// reducir modulo nada (ej: contar pares, hasta ~1e14 de resultado).
// Precision: es exacta si los coeficientes del resultado son menores
// a ~1e14 (regla practica: n * maxA * maxB < 1e14 aprox).
// Si el resultado es mayor, usar NTT con 2 modulos + CRT (ver 2_ntt.cpp)
// o multiplicar_mod mas abajo si te piden un modulo cualquiera.
const double PI = acos(-1.0);

// Transformada in-place. a.size() debe ser potencia de 2.
// invertir = false: coeficientes -> valores
// invertir = true : valores -> coeficientes (divide entre n al final)
// Complejidad: O(n log n)
void fft(vector<cd>& a, bool invertir) {
        int n = a.size();
        // 1) Permutacion bit-reversal
        for (int i = 1, j = 0; i < n; i++) {
                int bit = n >> 1;
                for (; j & bit; bit >>= 1) j ^= bit;
                j ^= bit;
                if (i < j) swap(a[i], a[j]);
        }
        // 2) Tabla de raices n-esimas de la unidad calculadas directamente
        //    con cos/sin (mas precisas que ir multiplicando w *= wl).
        vector<cd> w(n / 2);
        for (int i = 0; i < n / 2; i++) {
                double ang = 2 * PI * i / n;
```

```
                w[i] = cd(cos(ang), invertir ? -sin(ang) : sin(ang));
        }
        // 3) Mariposas
        for (int len = 2; len <= n; len <<= 1) {
                int paso = n / len; // la raiz len-esima es w[j * paso]
                for (int i = 0; i < n; i += len) {
                        for (int j = 0; j < len / 2; j++) {
                                cd u = a[i + j];
                                cd v = a[i + j + len / 2] * w[j * paso];
                                a[i + j] = u + v;
                                a[i + j + len / 2] = u - v;
                        }
                }
        }
        // 4) La inversa divide entre n
        if (invertir) for (auto& x : a) x /= n;
}

// Producto de polinomios con coeficientes enteros (pueden ser negativos).
// Resultado de tamanho a.size() + b.size() - 1. O((n+m) log(n+m))
vector<ll> multiply(const vector<ll>& a, const vector<ll>& b) {
        if (a.empty() || b.empty()) return {};
        int need = a.size() + b.size() - 1;
        int n = 1;
        while (n < need) n <<= 1;
        vector<cd> fa(a.begin(), a.end()), fb(b.begin(), b.end());
        fa.resize(n);
        fb.resize(n);
        fft(fa, false);
        fft(fb, false);
        for (int i = 0; i < n; i++) fa[i] *= fb[i];
        fft(fa, true);
        vector<ll> res(need);
        for (int i = 0; i < need; i++) res[i] = llround(fa[i].real()); // redondear
        return res;
}

// Producto modulo un modulo CUALQUIERA (ej: 1e9+7). Requiere 0 <= a[i] < mod <= 2^30.
// Idea: partir cada coeficiente en dos mitades de 15 bits para que los
// productos intermedios sean chicos y los doubles no pierdan precision.
//   a = a1*2^15 + a0,  b = b1*2^15 + b0
//   a*b = a1*b1*2^30 + (a0*b1 + a1*b0)*2^15 + a0*b0
vector<ll> multiplicar_mod(const vector<ll>& a, const vector<ll>& b, ll mod) {
        if (a.empty() || b.empty()) return {};
        int need = a.size() + b.size() - 1;
        int n = 1;
        while (n < need) n <<= 1;
        vector<cd> a0(n), a1(n), b0(n), b1(n);
        for (int i = 0; i < (int)a.size(); i++) a0[i] = a[i] & 32767, a1[i] = a[i] >> 15;
        for (int i = 0; i < (int)b.size(); i++) b0[i] = b[i] & 32767, b1[i] = b[i] >> 15;
        fft(a0, false); fft(a1, false);
        fft(b0, false); fft(b1, false);
        vector<cd> r00(n), r01(n), r11(n);
        for (int i = 0; i < n; i++) {
                r00[i] = a0[i] * b0[i];
                r01[i] = a0[i] * b1[i] + a1[i] * b0[i];
                r11[i] = a1[i] * b1[i];
        }
        fft(r00, true); fft(r01, true); fft(r11, true);
        vector<ll> res(need);
        for (int i = 0; i < need; i++) {
                ll x00 = llround(r00[i].real()) % mod;
                ll x01 = llround(r01[i].real()) % mod;
                ll x11 = llround(r11[i].real()) % mod;
                res[i] = (x00 + (x01 << 15) % mod + (x11 << 30) % mod) % mod;
        }
        return res;
}
```

7.3. Fht

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
typedef vector<ll> vl;

//  FHT: Fast Hadamard / Walsh-Hadamard Transform
// Convoluciones sobre MASCARAS DE BITS (en vez de suma de indices):
//   XOR: c[k] = sum de a[i]*b[j] con i ^ j = k
//   AND: c[k] = sum de a[i]*b[j] con i & j = k
//   OR : c[k] = sum de a[i]*b[j] con i | j = k
// El tamanho n debe ser potencia de 2 (n = 2^bits). Complejidad O(n log n).
// Todo se hace modulo MOD. Si necesitas valores exactos (sin modulo),
// quita los "% MOD" y en la inversa XOR divide directo entre n.
```

```cpp
const ll MOD = 1000000007;

ll pot(ll b, ll e) {
        ll res = 1;
        b %= MOD;
        while (e) {
                if (e & 1) res = res * b % MOD;
                b = b * b % MOD;
                e >>= 1;
        }
        return res;
}

// ---------- XOR ----------
// La mariposa (u, v) -> (u+v, u-v) es la transformada de Walsh-Hadamard.
// La inversa es la misma operacion y luego dividir entre n.
void fht_xor(vl& a, bool invertir) {
        int n = a.size();
        for (int len = 1; len < n; len <<= 1) {
                for (int i = 0; i < n; i += 2 * len) {
                        for (int j = 0; j < len; j++) {
                                ll u = a[i + j], v = a[i + j + len];
                                a[i + j] = (u + v) % MOD;
                                a[i + j + len] = (u - v + MOD) % MOD;
                        }
                }
        }
        if (invertir) {
                ll ninv = pot(n, MOD - 2);
                for (auto& x : a) x = x * ninv % MOD;
        }
}

// ---------- AND ----------
// Transformada zeta sobre SUPERCONJUNTOS: f^[S] = suma de f[T] con T contiene a S.
// Luego AND-convolucion = producto punto a punto.
// La inversa resta en vez de sumar (Mobius).
void fht_and(vl& a, bool invertir) {
        int n = a.size();
        for (int len = 1; len < n; len <<= 1) {
                for (int i = 0; i < n; i += 2 * len) {
                        for (int j = 0; j < len; j++) {
                                ll& u = a[i + j];
                                ll v = a[i + j + len];
                                if (!invertir) u = (u + v) % MOD;
                                else u = (u - v + MOD) % MOD;
                        }
                }
        }
}

// ---------- OR ----------
// Transformada zeta sobre SUBCONJUNTOS: f^[S] = suma de f[T] con T dentro de S.
// Luego OR-convolucion = producto punto a punto.
void fht_or(vl& a, bool invertir) {
        int n = a.size();
        for (int len = 1; len < n; len <<= 1) {
                for (int i = 0; i < n; i += 2 * len) {
                        for (int j = 0; j < len; j++) {
                                ll u = a[i + j];
                                ll& v = a[i + j + len];
                                if (!invertir) v = (v + u) % MOD;
                                else v = (v - u + MOD) % MOD;
                        }
                }
        }
}

// ---------- Convoluciones completas ----------
// a y b deben tener el mismo tamanho (potencia de 2).
vl conv_xor(vl a, vl b) {
        fht_xor(a, false);
        fht_xor(b, false);
        for (int i = 0; i < (int)a.size(); i++) a[i] = a[i] * b[i] % MOD;
        fht_xor(a, true);
        return a;
}

vl conv_and(vl a, vl b) {
        fht_and(a, false);
        fht_and(b, false);
        for (int i = 0; i < (int)a.size(); i++) a[i] = a[i] * b[i] % MOD;
        fht_and(a, true);
        return a;
}

```

```
vl conv_or(vl a, vl b) {
        fht_or(a, false);
        fht_or(b, false);
        for (int i = 0; i < (int)a.size(); i++) a[i] = a[i] * b[i] % MOD;
        fht_or(a, true);
        return a;
}

// Potencia de una familia bajo XOR-convolucion (a^k): transformar una vez,
// elevar cada posicion a k y volver. O(n log n + n log k)
vl potencia_xor(vl a, ll k) {
        fht_xor(a, false);
        for (auto& x : a) x = pot(x, k);
        fht_xor(a, true);
        return a;
}

// ============================================================
//  Ejemplo: Sum the Fibonacci (914G)
// ============================================================
// Con 17 bits, n = 1 << 17. Se arman tres arreglos indexados por mascara
// (pares (a,b) con a&b=0 agrupados por a|b; elementos simples; pares (d,e)
// agrupados por xor) y se combinan con conv_and. Luego solo importan las
// mascaras con un unico bit, a las que se les multiplica el Fibonacci.
```

7.4. Polinomios

```
//  OPERACIONES DE POLINOMIOS (modulo MOD = 998244353)
// REQUIERE: pegar este archivo DEBAJO de ntt (usa poly, MOD,
// addmod, submod, mulmod, pot, inv y multiply).
// Un poly es un vector donde p[i] es el coeficiente de x^i.

// Quita ceros al final (deja al menos 1 coeficiente)
void recortar(poly& p) {
        while (p.size() > 1 && p.back() == 0) p.pop_back();
}

poly sumar(const poly& a, const poly& b) {
        poly r(max(a.size(), b.size()));
        for (int i = 0; i < (int)a.size(); i++) r[i] = addmod(r[i], a[i]);
        for (int i = 0; i < (int)b.size(); i++) r[i] = addmod(r[i], b[i]);
        recortar(r);
        return r;
}

poly restar(const poly& a, const poly& b) {
        poly r(max(a.size(), b.size()));
        for (int i = 0; i < (int)a.size(); i++) r[i] = addmod(r[i], a[i]);
        for (int i = 0; i < (int)b.size(); i++) r[i] = submod(r[i], b[i]);
        recortar(r);
        return r;
}

// Derivada formal. Si p es constante devuelve {0}.
poly derivar(const poly& p) {
        if (p.size() <= 1) return {0};
        poly r(p.size() - 1);
        for (int i = 1; i < (int)p.size(); i++) r[i - 1] = mulmod(p[i], i);
        return r;
}

// Integral formal (constante de integracion = 0). Devuelve tamanho p.size()+1.
poly integrar(const poly& p) {
        int n = p.size();
        poly r(n + 1, 0);
        vector<ll> iv(n + 2, 1); // iv[i] = inverso de i, en O(n)
        for (int i = 2; i <= n; i++) iv[i] = (MOD - (MOD / i) * iv[MOD % i] % MOD) % MOD;
        for (int i = 0; i < n; i++) r[i + 1] = mulmod(p[i], iv[i + 1]);
        return r;
}

// Inverso multiplicativo de b modulo x^n: devuelve c (tamanho n) con b*c = 1 (mod x^n).
// Requiere b[0] != 0. Newton: c <- c * (2 - b*c); cada paso duplica la precision.
poly inversa(const poly& b, int n) {
        poly c = {inv(b[0])};
        for (int len = 1; len < n; len <<= 1) {
                int j = 2 * len;
                poly bb(b);
                bb.resize(j); // solo importan los primeros j coeficientes de b
                poly cb = multiply(c, bb);
                cb.resize(j);
                for (auto& x : cb) x = submod(0, x); // -b*c
                cb[0] = addmod(cb[0], 2);            // 2 - b*c
                c = multiply(c, cb);
```

```
                c.resize(j);
        }
        c.resize(n);
        return c;
}

// Division escolar O(n*m): devuelve {cociente, resto}. Rapida si los grados son chicos.
pair<poly, poly> divslow(const poly& a, const poly& b) {
        poly q, r = a;
        ll ib = inv(b.back());
        while (r.size() >= b.size()) {
                ll c = mulmod(r.back(), ib);
                q.push_back(c);
                if (c) {
                        for (int i = 0; i < (int)b.size(); i++) {
                                int pos = r.size() - 1 - i;
                                r[pos] = submod(r[pos], mulmod(c, b[b.size() - 1 - i]));
                        }
                }
                r.pop_back();
        }
        reverse(q.begin(), q.end());
        return {q, r};
}

// Division de polinomios: a = q*b + r, con grado(r) < grado(b).
// Devuelve {q, r}. b debe tener coeficiente lider distinto de 0.
// Idea: invirtiendo los coeficientes, q = rev(a) * inversa(rev(b)) mod x^(m-n+1).
pair<poly, poly> dividir(const poly& a, const poly& b) {
        int m = a.size(), n = b.size();
        const int MAGIC = 750; // bajo este umbral la division escolar es mas rapida
        if (m < n) return {{0}, a};
        if (min(m - n, n) < MAGIC) return divslow(a, b);
        poly ap(a), bp(b);
        reverse(ap.begin(), ap.end());
        reverse(bp.begin(), bp.end());
        bp = inversa(bp, m - n + 1);
        poly q = multiply(ap, bp);
        q.resize(m - n + 1);
        reverse(q.begin(), q.end());
        poly r = restar(a, multiply(b, q));
        return {q, r};
}

//  Logaritmo, exponencial y potencia de series de potencias

// ln(a) mod x^n. Requiere a[0] == 1.   ln(a)' = a'/a
poly logaritmo(const poly& a, int n) {
        poly r = multiply(derivar(a), inversa(a, n));
        r.resize(n - 1);
        r = integrar(r); // tamanho n
        return r;
}

// exp(a) mod x^n. Requiere a[0] == 0.
// Newton: g <- g * (1 - ln(g) + a)
poly exponencial(const poly& a, int n) {
        poly g = {1};
        for (int len = 1; len < n; len <<= 1) {
                int j = 2 * len;
                poly lg = logaritmo(g, j);
                poly t(j);
                for (int i = 0; i < j; i++) {
                        ll ai = i < (int)a.size() ? a[i] : 0;
                        ll li = i < (int)lg.size() ? lg[i] : 0;
                        t[i] = submod(ai, li);
                }
                t[0] = addmod(t[0], 1);
                g = multiply(g, t);
                g.resize(j);
        }
        g.resize(n);
        return g;
}

// a^k mod x^n, con k hasta 1e18. Funciona aunque a[0] == 0.
// Se factoriza a = c * x^s * b con b[0] = 1, y a^k = c^k * x^(s*k) * exp(k * ln b).
poly potencia(const poly& a, ll k, int n) {
        poly res(n, 0);
        if (k == 0) { res[0] = 1; return res; }
        int s = 0;
        while (s < (int)a.size() && a[s] == 0) s++;
        if (s == (int)a.size()) return res;           // a = 0
        if (s > 0 && k > (n - 1) / s) return res;     // s*k >= n: todo se sale de x^n
        int desp = s * k;                             // desplazamiento (< n)
        int m = n - desp;                             // coeficientes que realmente se necesitan
```

```
        ll c = a[s], ic = inv(c);
        poly b(m, 0);
        for (int i = 0; i < m; i++) if (s + i < (int)a.size()) b[i] = mulmod(a[s + i], ic);
        poly l = logaritmo(b, m);
        ll kk = k % MOD;
        for (auto& x : l) x = mulmod(x, kk);
        poly e = exponencial(l, m);
        ll ck = pot(c, k);
        for (int i = 0; i < m; i++) res[desp + i] = mulmod(e[i], ck);
        return res;
}

//  Evaluacion multipunto e interpolacion
// Arbol de productos: la hoja i es (x - x_i); cada nodo es el producto de
// sus dos hijos. arbol[1] = (x-x_0)(x-x_1)...(x-x_{k-1}).
vector<poly> arbol;

void construir_arbol(const vector<ll>& x) {
        int k = x.size();
        arbol.assign(2 * k, poly());
        for (int i = k; i < 2 * k; i++) arbol[i] = {submod(0, x[i - k] % MOD), 1};
        for (int i = k - 1; i >= 1; i--) arbol[i] = multiply(arbol[2 * i], arbol[2 * i + 1]);
}

// Usa el arbol ya construido. Baja tomando restos: P mod (x - x_i) = P(x_i).
vector<ll> evaluar_arbol(const poly& a, int k) {
        vector<poly> ans(2 * k);
        ans[1] = dividir(a, arbol[1]).second;
        for (int i = 2; i < 2 * k; i++) ans[i] = dividir(ans[i >> 1], arbol[i]).second;
        vector<ll> r(k);
        for (int i = 0; i < k; i++) r[i] = ans[i + k][0];
        return r;
}

// Evalua a en todos los puntos x. O(n log^2 n)
vector<ll> evaluar(const poly& a, const vector<ll>& x) {
        construir_arbol(x);
        return evaluar_arbol(a, x.size());
}

// Interpolacion de Lagrange: dado (x_i, y_i) con x_i distintos, devuelve
// el unico polinomio de grado < k que pasa por todos los puntos.
//   P(x) = sum_i y_i / M'(x_i) * M(x)/(x - x_i),   M = prod (x - x_j)
poly interpolar(const vector<ll>& x, const vector<ll>& y) {
        int k = x.size();
        construir_arbol(x);
        vector<ll> d = evaluar_arbol(derivar(arbol[1]), k); // M'(x_i)
        vector<poly> parcial(2 * k);
        for (int i = k; i < 2 * k; i++) parcial[i] = {mulmod(y[i - k] % MOD, inv(d[i - k]))};
        for (int i = k - 1; i >= 1; i--) {
                poly p1 = multiply(arbol[2 * i], parcial[2 * i + 1]);
                poly p2 = multiply(arbol[2 * i + 1], parcial[2 * i]);
                parcial[i] = sumar(p1, p2);
        }
        return parcial[1];
}
```

8. Flow

8.1. Dinic

```
struct Dinic
{
    const int INF = 1e9;

    struct flowEdge
    {
        int to, rev, f, cap;
    };

    vector<vector<flowEdge>> G;
    vector<int> work, lvl, Q;

    int S, T, nodes;

    Dinic(){}
    Dinic(int n, int s, int t){
        S = s;
        T = t;
        nodes = n;
        G.clear();
```

```
        G.resize(n);
        Q.resize(n);
    }
    void addEdge(int st, int en , int cap){
        flowEdge A = {en, (int)G[en].size(), 0, cap};
        flowEdge B = {st, (int)G[st].size(), 0, 0};
        G[st].push_back(A);
        G[en].push_back(B);
    }
    int dfs(int v, int f){
        if(v == T or f == 0) return f;
        for(int &i = work[v]; i < G[v].size(); i++){
            flowEdge &e = G[v][i];
            int u = e.to;
            if(e.cap <= e.f or lvl[u] != lvl[v] + 1) continue;
            int df = dfs(u, min(f, e.cap - e.f));
            if(df){
                e.f += df;
                G[u][e.rev].f -= df;
                return df;
            }
        }
        return 0;
    }
    bool bfs(){
        int qt = 0;
        Q[qt++] = S;
        lvl.assign(nodes,-1);
        lvl[S] = 0;
        for(int i = 0; i < qt; i++){
            int u = Q[i];
            for(flowEdge &e : G[u]){
                int v = e.to;
                if(e.cap <= e.f or lvl[v] != -1) continue;
                lvl[v] = lvl[u] + 1;
                Q[qt++] = v;
            }
        }
        return lvl[T] != -1;
    }
    int maxFlow(){
        int flow = 0;
        while(bfs()){
            work.assign(nodes,0);
            while(true){
                int df = dfs(S,INF);
                if(df == 0) break;
                flow += df;
            }
        }
        return flow;
    }

};
```

9. Graph Algorithms

9.1. Dijkstra

```
const int N = 1e5;

int vis[N];
vector<pair<int,int>> G[N];

void dijkstra(int x){
    priority_queue<pair<int,int>, vector<pair<int,int>>, greater<pair<int,int>>> pq;
    pq.push({0,x});
    vis[x] = 0;
    while(!pq.empty()){
        auto y = pq.top();
        pq.pop();
        int u = y.second;
        int cost = y.first;
        for(auto v : G[u]){
            int cur = cost + v.second;
            if(cur < vis[v.first]){
                vis[v.first] = cur;
                pq.push({cur,v.first});
            }
        }
    }
```

```
}
```

9.2. Kruskal Mst

```
const int N = 1e5;
struct DSU{
    int num;
    vector<int> parent;
    vector<int> sizes;
    DSU(){};
    DSU(int n){
        init(n);
    }
    void init(int n){
        num = n;
        parent.resize(n+1);
        sizes.resize(n+1);
        for(int i = 1; i <= n ; i++){
            parent[i] = i;
            sizes[i] = 1;
        }
    }
    int find(int a){
        if(a == parent[a]) return a;
        return parent[a] = find(parent[a]);
    }
    void join(int a, int b){
        int x = find(a);
        int y = find(b);
        if(x == y) return;
        num--;
        if(sizes[x] < sizes[y]){
            parent[x] = y;
            sizes[y]+=sizes[x];
        }
        else{
            parent[y] = x;
            sizes[x]+=sizes[y];
        }
    }
};
int n;
vector<pair<int,pair<int,int>>> List;
int Kruskal_MST(){
    sort(List.begin(),List.end());
    DSU dsu(n);
    int mst = 0, t = 1;
    for(int i = 0; i < List.size(); i++){
        int w = List[i].first; //weigth
        int a = List[i].second.first; //at
        int b = List[i].second.second; //to
        if(dsu.find(a) != dsu.find(b)){
            dsu.join(a,b);
            mst+=w;
            t++;
        }
        if(t == n) break;
    }
    return mst;
}
```

9.3. Topological Sort Kahn Algorithm

```
const int N = 1e5;

int indegree[N];
vector<int> G[N];
int n;
vector<int> Kahn(){
    queue<int> q;
    vector<int> ans;
    for(int i = 0; i < n; i++){
        if(indegree[i] == 0) q.push(i);
    }
    while (!q.empty()){
        int v = q.front();
        q.pop();
        ans.push_back(v);
        for(int u : G[v]){
            indegree[u]--;
            if(indegree[u] == 0){
```

```
                q.push(u);
            }
        }
    }
    //if(ans.size() != n) grafo no DAG
    return ans;
}
```

9.4. Kosaraju

```
struct Kosaraju {
    int n, components;
    vector<vector<int>> graph, invgraph;
    vector<int> vis, order, component;

    Kosaraju(int n) : n(n) {
        graph.resize(n + 1);
        invgraph.resize(n + 1);
        vis.resize(n + 1);
        component.resize(n + 1);
    }

    void add_edge(int u, int v) {
        graph[u].push_back(v);
        invgraph[v].push_back(u);
    }

    void dfs1(int u) {
        vis[u] = 1;
        for (int v : graph[u]) {
            if (!vis[v])
                dfs1(v);
        }
        order.push_back(u);
    }

    void dfs2(int u, int id) {
        vis[u] = 1;
        component[u] = id;
        for (int v : invgraph[u]) {
            if (!vis[v])
                dfs2(v, id);
        }
    }
    void build() {
        for (int u = 1; u <= n; u++) {
            if (!vis[u])
                dfs1(u);
        }
        reverse(order.begin(), order.end());
        fill(vis.begin(), vis.end(), 0);
        components = 0;
        for (int u : order) {
            if (!vis[u]) {
                components++;
                dfs2(u, components);
            }
        }
    }
};
```

9.5. Bridges

```
vector<vector<int>> g;
vector<int> tin, low;
vector<bool> vis;
vector<pair<int,int>> bridges;

int timer = 0;

void dfs(int u, int p = -1){
    vis[u] = true;
    tin[u] = low[u] = timer++;

    for(int v : g[u]){
        if(v == p) continue;

        if(vis[v]){
            // Arista hacia un ancestro
            low[u] = min(low[u], tin[v]);
        }
```

```
        else{
            dfs(v, u);

            // El sub rbol de v puede llegar hasta...
            low[u] = min(low[u], low[v]);

            // Si no puede regresar a u ni a un ancestro de u,
            // entonces (u,v) es puente
            if(low[v] > tin[u]){
                bridges.push_back({u, v});
            }
        }
    }
}
```

9.6. Kuhn

```
int n;
vector<vector<int>> G;
vector<int> mt;
vector<bool> used;
bool try_kuhn(int v){
    if(used[v]) return false;
    used[v] = true;
    for(int to : G[v]){
        if(mt[to] == -1 or try_kuhn(mt[to])){
            mt[to] = v;
            return true;
        }
    }
    return false;
}
void solve(int tc){
    mt.assign(n, -1);
    repl(v,0,n){
        used.assign(n, false);
        try_kuhn(v);
    }
}
```

9.7. 2sat

```
struct TwoSAT {
    int n, nodes;
    vector<vector<int>> graph, invgraph;
    vector<int> vis, order, component;
    vector<bool> ans;

    TwoSAT(int n) : n(n), nodes(2 * n) {
        graph.resize(nodes);
        invgraph.resize(nodes);
        vis.resize(nodes);
        component.resize(nodes);
        ans.resize(n);
    }

    int id(int x, bool value) {
        return 2 * x + value;
    }

    int neg(int x) {
        return x ^ 1;
    }

    void add_implication(int a, int b) {
        graph[a].push_back(b);
        invgraph[b].push_back(a);
    }

    void add_or(int a, int b) {
        add_implication(neg(a), b);
        add_implication(neg(b), a);
    }

    void add_xor(int a, int b) {
        add_or(a, b);
        add_or(neg(a), neg(b));
    }

    void add_equivalence(int a, int b) {
        add_implication(a, b);
```

```
        add_implication(b, a);
        add_implication(neg(a), neg(b));
        add_implication(neg(b), neg(a));
    }

    void dfs1(int u) {
        vis[u] = 1;
        for (int v : graph[u]) {
            if (!vis[v])
                dfs1(v);
        }
        order.push_back(u);
    }

    void dfs2(int u, int id) {
        vis[u] = 1;
        component[u] = id;
        for (int v : invgraph[u]) {
            if (!vis[v])
                dfs2(v, id);
        }
    }

    bool solve() {
        for (int u = 0; u < nodes; u++) {
            if (!vis[u])
                dfs1(u);
        }
        reverse(order.begin(), order.end());
        fill(vis.begin(), vis.end(), 0);
        int cnt = 0;
        for (int u : order) {
            if (!vis[u]) {
                dfs2(u, cnt++);
            }
        }
        for (int i = 0; i < n; i++) {
            if (component[2 * i] == component[2 * i + 1])
                return false;
            ans[i] = component[2 * i] > component[2 * i + 1];
        }
        return true;
    }
};
```

10. Tree Algorithms

10.1. Prim

```
// mst con prim
vector<ll> dist(n,inf);
vector<int> vis(n,0);
dist[0] = 0;
for(int i = 0 ; i < n ; i ++){
        int u = -1;
        for(int j = 0; j < n; j ++){
                if(!vis[j] and (u == -1 or dist[j] < dist[u]))u = j;
        }
        vis[u] = 1;
        for(int j = 0 ; j < n ; j++){
                if(vis[j])continue;
                dist[j] = min(dist[j], v[u].dist(v[j]));
        }
}
```

10.2. Euler Tour

```
const int N = 1e5 + 10;
vector<int> path;
vector<int> G[N];
void EulerTour(int node, int parent){
    path.pb(node);
    for(int u : G[node]){
        if(u != parent){
            EulerTour(u,node);
        }
    }
    path.pb(node);
```

```
}

const int N = 1e5 + 10;
int tin[N];
int tout[N];
int flat[N];
int timer = 1;
void EulerTour(int node, int parent){
    tin[node] = timer;
    flat[timer] = node;
    timer++;
    for(int u : G[node]){
        if(u != parent){
            EulerTour(u, node);
        }
    }
    tout[node] = timer;
}
```

10.3. Binary Lifting

```
const int N = 1e5;
const int LOG = 20;
int up[N][LOG];
vector<int> G[N];
//Find parents
void dfs(int u){
    for(int v : G[u]){
        up[v][0] = u;
        for(int j = 1; j < LOG; j++){
            up[v][j] = up[up[v][j-1]][j-1];
        }
        dfs(v);
    }
}
int get_K_parent(int a, int k){
    for(int j = LOG-1; j >= 0; j++){
        if(k&(1<<j)){
            a = up[a][j];
        }
    }
    return a;
}
```

10.4. Binary Lifting2

```
const int LOG = 20;

int up[N][LOG];

//ya tenemos preprocesada la informacion de los padres directos
//up[v][0] ya contiene el padre de v
//no importa el orden

for(int j = 1; j < LOG; j++){

    for(int v = 1; v <= n; v++){

        up[v][j] = up[up[v][j-1]][j-1];

    }

}
```

10.5. Centroid Decomposition

```
//EJERCICIO IMPLEMENTACION XENIA AND TREE
//Nodo rojo mas cercano a un nodo v

const int N = 2e5+10;
vector<int>G[N];
int tag[N]; // time of discovery of centrodi
int fat[N]; // father in centroid decomposition
int szt[N]; // size of subtree

int best[N];
int niv[N];
```

```cpp
int dist[30][N]; //dist[nivel][nodo];

void dfs_level(int node, int p, int niv, int prof){
    dist[niv][node] = prof;
    for(auto y : G[node]){
        if(y != p and tag[y] < 0){
            dfs_level(y, node, niv, prof+1);
        }
    }
}
// -----------------CENTROID-------------------------
int calcsz(int node, int p){
    szt[node] = 1;
    for(auto y : G[node]){
        if(y != p and tag[y] < 0){
            szt[node] += calcsz(y, node);
        }
    }
    return szt[node];
}
int ccnt = 0;
void cdfs(int node, int p, int nivel = 0, int sz = -1){
    if(sz < 0) sz = calcsz(node, -1);
    for(auto y : G[node]){
        if(tag[y] < 0 and szt[y] * 2 >= sz){
            szt[node] = 0;
            cdfs(y, p, nivel, sz);
            return;
        }
    }
    tag[node] = ccnt++;
    fat[node] = p;
    niv[node] = nivel;
    dfs_level(node, -1, nivel, 0);
    for(auto y : G[node]){
        if(tag[y] < 0){
            cdfs(y, node, nivel+1);
        }
    }
}

int n;
void centroid(){
    ccnt = 0;
    repl(i,1,n+1){
        tag[i] = -1;
    }
    cdfs(1, -1);
}

//-----------------------------------------------

void update(int node, int rojo){
    int nivel = niv[node];
    int cost = dist[nivel][rojo];
    best[node] = min(best[node], cost);
    int p = fat[node];
    if(p > 0){
        update(p, rojo);
    }
}

int improve(int node, int cur){
    if(node <= 0){
        return 1e9;
    }
    int nivel = niv[node];
    int cost = best[node] + dist[nivel][cur];
    int p = fat[node];
    return min(cost, improve(p, cur));
}

void solve(){
    int qq;
    cin >> n >> qq;
    repl(i,0,30){
        repl(j,1,n+1){
            dist[i][j] = -1;
            best[j] = 1e9;
        }
    }
    repl(i,0,n-1){
        int a, b;
        cin >> a >> b;
        G[a].push_back(b);
        G[b].push_back(a);
```

```
    }
    centroid();
    update(1, 1);
    while(qq--){
        int q, node;
        cin >> q >> node;
        if(q == 1){
            update(node, node);
        }
        else{
            int res = best[node];
            int res2 = improve(node, node);
            cout << min(res, res2) << '\n';
        }
    }
}

int main(){
    velocito;
    int t = 1;
    // cin >> t;
    while(t--){
        solve();
    }
    return 0;
}
//Sabrossus
//TC Argentina
```

10.6. Chu Liu Edmonds

```
//minimo costo de un mst dirigido:

struct Edge {
    int u, v;
    ll w;
};

ll chu_liu_edmonds(int n, int root, vector<Edge> edges) {
    ll ans = 0;
    while (true) {
        vector<ll> in(n, inf);
        vector<int> pre(n, -1);
        for (auto e : edges) {
            if (e.u != e.v and e.v != root and e.w < in[e.v]) {
                in[e.v] = e.w;
                pre[e.v] = e.u;
            }
        }
        for (int i = 0; i < n; i++) {
            if (i == root) continue;
            if (in[i] == inf) return -1;
        }
        for (int i = 0; i < n; i++) {
            if (i != root) ans += in[i];
        }
        int cnt = 0;
        vector<int> id(n, -1), vis(n, -1);
        for (int i = 0; i < n; i++) {
            int v = i;
            while (v != root and vis[v] != i and id[v] == -1) {
                vis[v] = i;
                v = pre[v];
            }
            if (v != root and id[v] == -1) {
                for (int u = pre[v]; u != v; u = pre[u]) {
                    id[u] = cnt;
                }
                id[v] = cnt++;
            }
        }
        if (cnt == 0) break;
        for (int i = 0; i < n; i++) {
            if (id[i] == -1) id[i] = cnt++;
        }
        vector<Edge> nxt;
        for (auto e : edges) {
            int u = id[e.u];
            int v = id[e.v];
            ll w = e.w;
            if (u != v) {
                if (id[e.v] != id[e.u]) {
                    w -= in[e.v];
                }
```

```
                nxt.push_back({u, v, w});
            }
        }
        root = id[root];
        n = cnt;
        edges = move(nxt);
    }
    return ans;
}
```

10.7. Chu Liu Edmonds Construction

```
// construccion de mst dirigido

const ll inf = 1e18;

struct ChuLiuEdmonds {
    struct Edge {
        int u, v;
        ll w;
    };

    struct Result {
        bool ok;
        ll cost;
        vector<Edge> mst;
    };

    int n;
    vector<Edge> edges;

    ChuLiuEdmonds(int n) : n(n) {}

    void add_edge(int u, int v, ll w) {
        edges.push_back({u - 1, v - 1, w});
    }

    struct E {
        int u, v;
        ll w;
        int id;
    };

    struct Sol {
        bool ok;
        vector<int> chosen;
    };

    Sol dfs(int n, int root, vector<E>& edges) {
        vector<ll> in(n, inf);
        vector<int> pre(n, -1), id(n, -1), vis(n, -1);
        for (int i = 0; i < (int)edges.size(); i++) {
            auto e = edges[i];
            if (e.u != e.v and e.v != root and e.w < in[e.v]) {
                in[e.v] = e.w;
                pre[e.v] = e.u;
            }
        }
        in[root] = 0;
        for (int i = 0; i < n; i++) {
            if (in[i] == inf) return {false, {}};
        }
        int cnt = 0;
        for (int i = 0; i < n; i++) {
            int v = i;
            while (v != root and vis[v] != i and id[v] == -1) {
                vis[v] = i;
                v = pre[v];
            }
            if (v != root and id[v] == -1) {
                for (int u = pre[v]; u != v; u = pre[u]) {
                    id[u] = cnt;
                }
                id[v] = cnt++;
            }
        }
        if (cnt == 0) {
            vector<int> chosen;
            for (int i = 0; i < n; i++) {
                if (i != root) {
                    int idx = -1;
                    for (int j = 0; j < (int)edges.size(); j++) {
                        if (edges[j].v == i and
                            edges[j].u == pre[i] and
```

```
                            edges[j].w == in[i]) {
                            idx = j;
                            break;
                        }
                    }
                    chosen.push_back(idx);
                }
            }
            for (int x : chosen) {
                if (x == -1) return {false, {}};
            }
            return {true, chosen};
        }
        for (int i = 0; i < n; i++) {
            if (id[i] == -1) id[i] = cnt++;
        }
        vector<E> nxt;
        for (int i = 0; i < (int)edges.size(); i++) {
            auto e = edges[i];
            int u = id[e.u];
            int v = id[e.v];
            if (u != v) {
                nxt.push_back({u, v,e.w - in[e.v],i});
            }
        }
        Sol res = dfs(cnt, id[root], nxt);
        if (!res.ok) return {false, {}};
        vector<int> chosen(n, -1);
        for (int i = 0; i < n; i++) {
            if (i != root) {
                for (int j = 0; j < (int)edges.size(); j++) {
                    if (edges[j].v == i and
                        edges[j].u == pre[i] and
                        edges[j].w == in[i]) {
                        chosen[i] = j;
                        break;
                    }
                }
                if (chosen[i] == -1) return {false, {}};
            }
        }
        for (int j : res.chosen) {
            int old_idx = nxt[j].id;
            chosen[edges[old_idx].v] = old_idx;
        }
        vector<int> ans;
        for (int i = 0; i < n; i++) {
            if (i != root) {
                if (chosen[i] == -1) return {false, {}};
                ans.push_back(chosen[i]);
            }
        }
        return {true, ans};
    }

    Result get_mst(int root) {
        vector<E> e;
        for (int i = 0; i < (int)edges.size(); i++) {
            e.push_back({edges[i].u,edges[i].v,edges[i].w,i});
        }
        Sol res = dfs(n, root - 1, e);
        if (!res.ok) return {false, -1, {}};
        Result ans;
        ans.ok = true;
        ans.cost = 0;
        for (int i : res.chosen) {
            auto x = edges[i];
            ans.cost += x.w;
            ans.mst.push_back({x.u + 1, x.v + 1, x.w});
        }
        return ans;
    }

    bool possible(int root) {
        return get_mst(root).ok;
    }

    ll get_min_cost(int root) {
        Result res = get_mst(root);
        return res.ok ? res.cost : -1;
    }
};
```

10.8. Heavy Light Decomposition

```
struct HLD {
        int n, act;
        vector<vector<int>> adj;
        vector<int> parent, depth, heavy, head, pos;
        vector<int> val;
        vector<int> seg;
        int tam;
        HLD(int n) : n(n) {
                adj.resize(n+1);
                parent.resize(n+1);
                depth.resize(n+1);
                heavy.assign(n+1, -1);
                head.resize(n+1);
                pos.resize(n+1);
                val.resize(n+1);
                act = 0;
        }

        void add_edge(int u, int v) {
                adj[u].push_back(v);
                adj[v].push_back(u);
        }

        int dfs(int u, int p) {
                parent[u]=p;
                int size=1, max_sub=0;
                for (int v : adj[u]){
                        if(v==p)continue;
                        depth[v]=depth[u]+1;
                        int sub=dfs(v,u);
                        if(sub>max_sub){
                                max_sub=sub;
                                heavy[u]=v;
                        }
                        size+=sub;
                }
                return size;
        }

        void decompose(int u, int h) {
                head[u]=h;
                pos[u]=act++;
                if(heavy[u]!=-1)decompose(heavy[u], h);
                for(int v: adj[u]){
                        if(v!=parent[u] and v!=heavy[u])decompose(v,v);
                }
        }

        void build(){
                tam=1;
                while (tam < n) tam<<=1;
                seg.assign(2*tam,inf);
                for (int i = 1; i <= n; i++){
                        seg[tam+pos[i]]=val[i];
                }
                for (int i = tam - 1; i > 0; i--){
                        seg[i]=max(seg[2*i],seg[2*i+1]);
                }
        }

        void update(int u, int value) {
                int i=tam+pos[u];
                seg[i]=value;
                for(i>>=1; i; i>>=1){
                        seg[i]=max(seg[2*i],seg[2*i+1]);
                }
        }

        int query(int l, int r) {
                int res=inf;
                l+=tam;
                r+=tam;
                while(l <= r){
                        if(l&1)res=max(res,seg[l++]);
                        if(!(r&1))res=max(res,seg[r--]);
                        l>>=1;
                        r>>=1;
                }
                return res;
        }

        void init(){
                int root=1;
                dfs(root, 0);
```

```
                decompose(root,root);
                build();
        }

        int query_path(int a, int b){
                int res=inf;
                while(head[a]!=head[b]){
                        if(depth[head[a]]<depth[head[b]])swap(a,b);
                        res=max(res,query(pos[head[a]],pos[a]));
                        a=parent[head[a]];
                }
                if(depth[a]>depth[b])swap(a,b);
                res=max(res,query(pos[a],pos[b]));
                return res;
        }
};

void solve(){
        int n,q;
        cin>>n>>q;
        HLD hld(n);
        for(int i = 1 ; i <= n ; i++){
                cin>>hld.val[i];
        }
        for(int i = 0; i  < n-1 ; i++){
                int a,b;
                cin>>a>>b;
                hld.add_edge(a,b);
        }
        hld.init();
        while(q--){
                int t;
                cin>>t;
                if(t==1){
                        int s,x;
                        cin>>s>>x;
                        hld.update(s,x);
                }
                else{
                        int a,b;
                        cin>>a>>b;
                        cout<<hld.query_path(a,b)<<' ';
                }
        }
}
```

10.9. Lowest Common Ancestor

```
const int N = 1e5;
const int LOG = 20;
vector<int> G[N];
int depth[N];
int up[N][LOG];
// Assign levels and get parents
void dfs(int u){
    for(int v : G[u]){
        depth[v] = depth[u] + 1;
        up[v][0] = u;
        for(int j = 1; j < LOG; j++){
            up[v][j] = up[up[v][j-1]][j-1];
        }
        dfs(v);
    }
}
int get_lca(int a, int b){
    if(depth[a] < depth[b]) swap(a,b);
    int k = depth[a] - depth[b];
    for(int j = LOG-1; j >= 0; j--){
        if(k & (1<<j)){
            a = up[a][j];
        }
    }
    if(a == b) return a;
    for(int j = LOG-1; j >= 0; j--){
        if(up[a][j] != up[b][j]){
            a = up[a][j];
            b = up[b][j];
        }
    }
    return up[a][0];
}
```

11. Utilities

11.1. Prime Numbers

```
    2    3    5    7   11   13   17   19   23   29
   31   37   41   43   47   53   59   61   67   71
   73   79   83   89   97  101  103  107  109  113
  127  131  137  139  149  151  157  163  167  173
  179  181  191  193  197  199  211  223  227  229
  233  239  241  251  257  263  269  271  277  281
  283  293  307  311  313  317  331  337  347  349
  353  359  367  373  379  383  389  397  401  409
  419  421  431  433  439  443  449  457  461  463
  467  479  487  491  499  503  509  521  523  541
  547  557  563  569  571  577  587  593  599  601
  607  613  617  619  631  641  643  647  653  659
  661  673  677  683  691  701  709  719  727  733
  739  743  751  757  761  769  773  787  797  809
  811  821  823  827  829  839  853  857  859  863
  877  881  883  887  907  911  919  929  937  941
  947  953  967  971  977  983  991  997 1009 1013
 1019 1021 1031 1033 1039 1049 1051 1061 1063 1069
 1087 1091 1093 1097 1103 1109 1117 1123 1129 1151
 1153 1163 1171 1181 1187 1193 1201 1213 1217 1223
 1229 1231 1237 1249 1259 1277 1279 1283 1289 1291
 1297 1301 1303 1307 1319 1321 1327 1361 1367 1373
 1381 1399 1409 1423 1427 1429 1433 1439 1447 1451
 1453 1459 1471 1481 1483 1487 1489 1493 1499 1511
 1523 1531 1543 1549 1553 1559 1567 1571 1579 1583
 1597 1601 1607 1609 1613 1619 1621 1627 1637 1657
 1663 1667 1669 1693 1697 1699 1709 1721 1723 1733
 1741 1747 1753 1759 1777 1783 1787 1789 1801 1811
 1823 1831 1847 1861 1867 1871 1873 1877 1879 1889
 1901 1907 1913 1931 1933 1949 1951 1973 1979 1987

       970'997␣␣␣␣␣␣␣971'483   921'281'269   999'279'733
1'000'000'009␣1'000'000'021 1'000'000'409␣1'005'012'527
1000551285057160091l
```

11.2. 2D Prefix Sum

```
//1 indexed
const int N = 1005;
int v[N][N];
int acu[N][N];
int n, m;

void solve() {
    //limpiar la acu para cada TC

    // Construccion: acumula 1 si el elemento es exactamente 1
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            int val = (v[i][j] == 1 ? 1 : 0);
            acu[i][j] = val + acu[i - 1][j] + acu[i][j - 1] - acu[i - 1][j - 1];
        }
    }

    // Esquinas de consulta:
    // (a, b) -> Superior Izquierda
    // (c, d) -> Inferior Derecha
    int a = 2, b = 2, c = 4, d = 4; // Valores de ejemplo

    int ans = acu[c][d] - acu[a - 1][d] - acu[c][b - 1] + acu[a - 1][b - 1];
    cout << ans << '\n';
}
```

11.3. Ascii Table

```
ASCII QUICK REFERENCE
============================================================

CHARACTER RANGES
------------------------------------------------------------
'0' ... '9'     48 ... 57      -> c - '0'
'A' ... 'Z'     65 ... 90      -> c - 'A'
'a' ... 'z'     97 ... 122     -> c - 'a'

```

```
RELATIONSHIPS
------------------------------------------------------------
'a' - 'A' = 32
'A' + 32   = 'a'
'a' - 32   = 'A'

DIGITS
------------------------------------------------------------
'0' = 48        '5' = 53
'1' = 49        '6' = 54
'2' = 50        '7' = 55
'3' = 51        '8' = 56
'4' = 52        '9' = 57

UPPERCASE
------------------------------------------------------------
A 65   B 66   C 67   D 68   E 69   F 70   G 71
H 72   I 73   J 74   K 75   L 76   M 77   N 78
O 79   P 80   Q 81   R 82   S 83   T 84   U 85
V 86   W 87   X 88   Y 89   Z 90

LOWERCASE
------------------------------------------------------------
a 97   b 98   c 99   d 100  e 101  f 102  g 103
h 104  i 105  j 106  k 107  l 108  m 109  n 110
o 111  p 112  q 113  r 114  s 115  t 116  u 117
v 118  w 119  x 120  y 121  z 122

COMMON SYMBOLS
------------------------------------------------------------
SPACE  32       ! 33       " 34       # 35       $ 36
% 37       & 38       ' 39       ( 40       ) 41
* 42       + 43       , 44       - 45       . 46
/ 47       : 58       ; 59       < 60       = 61
> 62       ? 63       @ 64       [ 91       \ 92
] 93       ^ 94       _ 95       ` 96       { 123
| 124      } 125      ~ 126

USEFUL TRICKS
------------------------------------------------------------
int x = c - '0';        // digit -> integer
char c = '0' + x;       // integer -> digit

int x = c - 'a';        // lowercase -> [0,25]
int x = c - 'A';        // uppercase -> [0,25]

islower(c)
isupper(c)
isdigit(c)
isalpha(c)
isalnum(c)
isspace(c)
tolower(c)
toupper(c)
============================================================
```

11.4. Cmp for Sort

```cpp
//cambia Node por el tipo de dato
sort(a.begin(), a.end(), [](Node A, Node B){
  return A.c * B.w > B.c * A.w;
});
```

11.5. Int128

```cpp
// --- LECTURA B SICA ---
__int128 read128() {
    string s;
    cin >> s;
    __int128 n = 0;
    int sign = 1;
    size_t start = 0;

    if (!s.empty() && s[0] == '-') {
        sign = -1;
        start = 1;
    }

    for (size_t i = start; i < s.length(); ++i) {
        n = n * 10 + (s[i] - '0');
    }
```

```

    return n * sign;
}

// --- IMPRESI N B SICA ---
void print128(__int128 n) {
    if (n == 0) {
        cout << 0;
        return;
    }
    if (n < 0) {
        cout << '-';
        n = -n;
    }
    string s;
    while (n > 0) {
        s += (char)('0' + (n % 10));
        n /= 10;
    }
    reverse(s.begin(), s.end());
    cout << s;
}
```

11.6. Merge Sort

```
// Rango de uso: 0-indexed.
// Llamar en el solve como: mergeSort(v, 0, n - 1);
ll ans = 0;

void mergeSort(vector<ll> &v, int low, int high) {
    if (low == high) return;

    int mid = (low + high) >> 1;
    mergeSort(v, low, mid);
    mergeSort(v, mid + 1, high);

    queue<ll> L, H;
    for (int i = low; i < mid + 1; i++) {
        L.push(v[i]);
    }
    for (int i = mid + 1; i < high + 1; i++) {
        H.push(v[i]);
    }

    for (int i = low; i < high + 1; i++) {
        if (L.size() == 0) {
            v[i] = H.front();
            H.pop();
        }
        else if (H.size() == 0) {
            v[i] = L.front();
            L.pop();
        }
        else {
            if (L.front() <= H.front()) {
                v[i] = L.front();
                L.pop();
            }
            else {
                v[i] = H.front();
                H.pop();
                ans += L.size(); // Inversion detectada
            }
        }
    }
}
```

11.7. Pragma

```
#pragma GCC optimize("O3,unroll-loops")
#pragma GCC target("avx2,bmi,bmi2,lzcnt,popcnt")
```

11.8. Priority Queue Min Heap

```
priority_queue<int, vector<int>, greater<int>> pq;
```

11.9. Python Template

```python
import sys
from bisect import bisect_left, bisect_right, insort
from collections import defaultdict, Counter, deque
from heapq import heappush, heappop, heapify
from math import gcd, lcm, isqrt

input = sys.stdin.readline


def solve():
    n = int(input()) #leer numero
    a,b,c= map(int, input().split()) #leer numeros
    s = input().strip() #leer cadenas
    v = list(map(int, input().split())) #leer lista de enteros


t = 1
# t = int(input())
for _ in range(t):
    solve()

# C++                           Python

# vector                        list
# array                         list
# string                        str

# set                           set
# map                           dict
# multiset                      Counter
# unordered_map                 dict
# unordered_set                 set

# priority_queue                heapq
# queue                         deque
# deque                         deque

# lower_bound                   bisect_left
# upper_bound                   bisect_right
# sort                          list.sort()
# binary_search                 bisect_left/right

# pair                          tuple
# struct                        class / dataclass

# gcd                           math.gcd
# lcm                           math.lcm

# push_back                     append
# push_front                    appendleft
# pop_back                      pop
# pop_front                     popleft

# size()                        len()
# empty()                       not x
```

11.10. Random

```cpp
mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
// mt19937_64 para 64 bits usar uint64_t o unsigned long long
cout << rng() << endl;
shuffle(v.begin(),v.end(),rng);
```